

第一章：绪论

1 计算机程序设计语言的发展

1.1 概念

- 程序：指令集合，等于数据结构 + 算法 + 程序设计方法 + 语言工具；
- **源程序**经过**翻译程序**翻译后，最底层能被机器识别的**目标程序**是**机器代码**，翻译程序分为**汇编程序**（汇编语言翻译为机器语言）、**编译程序**（高级语言翻译为机器语言）、**解释程序**（边翻译边执行）。

1.2 计算机语言

- 机器语言：二进制语言，**机器唯一能听懂的语言**，属于低级语言；
- 汇编语言：符号化的机器指令，属于低级语言；
- 结构化高级语言：面向过程的编程（C语言、Basic等），属于高级语言；
- 非结构化高级语言：面向对象和可视化的编程（C++语言、Java等），属于高级语言。
- **需要区分：编译型语言、解释型语言。**

1.3 面向对象的语言

- 更直接地描述客观世界中存在的事物(对象)以及它们之间的关系，更直观！
- 将客观事物看作具有属性和方法的对象，抽象出类；
- 通过类的继承与多态实现代码重用，减少代码量。

2 面向对象的方法

2.1 面向过程的方法

- 特点：功能上划分了多个基本模块，**结构化程序设计**，形成了一个树状结构，由**顺序、选择、循环结构**组成；
- 缺点：可重用性差、数据安全性差、难以开发大型软件和图形界面的应用软件；
- 技术只是认知世界的工具，只是表征不是本征。

2.2 面向对象的方法

- 特点：对象 = 算法 + 数据结构；程序由对象组成，按照客观世界的描述建模；
- 优点：程序模块的独立性、数据的安全性就有了良好的保障。通过继承与多态性，可以大大提高程序的可重用性，使得软件的开发和维护都更为方便。

2.3 概念

- 对象：系统中用来描述客观事物的一个实体，对象由一组属性和一组行为（或方法）构成；
- 类：具有相同**属性**和**行为（方法）**的一组**对象**的集合；
- **消息**是描述事件发生的信息，它是对象之间发出的行为请求，多个消息构成一个**事件**；
- 消息的响应：对象执行操作被称为对消息的响应；
- **方法**包括**接口**和**方法体**两个部分；
- 基本特征：
 - 抽象性：对生活中的实体进行属性和行为上的抽象；
 - 封装性：把对象的属性和行为结合成一个独立的单位，并尽可能隐蔽对象的内部细节；
 - 继承性：指特殊类的对象拥有其一般类的属性和行为的特性，分为单继承、多继承，也可以分为取代继承、包含继承、受限继承（部分继承且没有添加新属性方法）、特化继承（修改了原有的属性方法）；
 - 多态性：指类中同一函数名对应多个具有相似功能的不同函数，包括编译时的多态（函数重载、运算符重载）、运行时的多态（虚函数）；

3 面向对象的软件开发

1. 面向对象的分析（OOA）：直接用问题域中客观存在的事物建立模型中的对象，对单个事物及事物之间的关系；
2. 面向对象的设计（OOD）：针对具体实现中的人机界面、数据存储、任务管理等因素补充一些与实现有关的部分；
3. 面向对象的编程（OOP）；
4. 面向对象的测试（OOT）：以对象的类作为基本测试单位，可以更准确地发现程序错误并提高测试效率；
5. 面向对象的软件维护（OOSM）：使用面向对象的方法开发的软件，其程序与问题域是一致的，因此，在维护阶段运用面向对象的方法可以大大提高软件维护的效率。

4 信息的表示和存储

4.1 信息的分类

- 控制信息：指令、控制字；
- 数据信息：数值信息（如定点数、浮点数）、非数值信息（如字符数据）。

4.2 进制转换

- R转十：各位数字与它的权相乘，其积相加；
- 十转R：整数“除R取余”法、小数“乘R取整”法；

4.3 存储单位

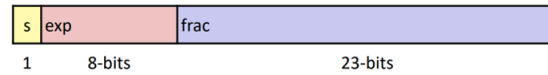
- 位、字节（KB,MB,GB,TB）、字；
- 原码：符号位0、1表示；
- 反码：原码符号位不变，其余取反；
- 补码：负数则反码的末位加1、正数和原码一致。

4.4 定点数与浮点数

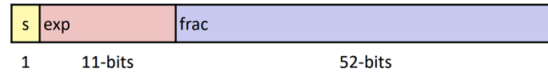
1. 一个M位整数(包括一位符号位):
 - (a) 采用原码或反码： $-(2^{m-1} - 1)$ 至 $2^{m-1} - 1$ ；
 - (b) 采用补码： -2^{m-1} 至 $2^{m-1} - 1$ 。
2. 浮点数：真实的IEEE表示方法如下，PPT中的exp部分用的是补码存储。

- 符号位 s 决定数字是负数还是正数
- 尾数 M 通常是一个范围为 $[1.0, 2.0)$ 内的小数
- 阶码 E 是 2 的幂, 表明权重

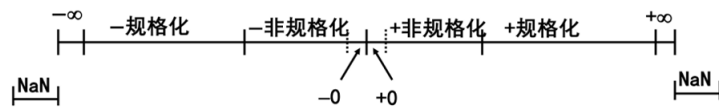
单精度: 32 bits



双精度: 64 bits



- 规格化的值 —— $\text{exp} \neq 0$ and $\text{exp} \neq 11\dots 11$
 $E = \text{exp} - \text{Bias}$
 $\text{Bias} = 2^{k-1} - 1$ 单精度: 127 双精度: 1023
 尾数编码, 有一个隐含的最高位 1
- 非规格化的值 —— $\text{exp} = 00\dots 00$
 $E = 1 - \text{Bias}$ 注: 此处不是 $0 - \text{Bias}$
 尾数编码, 有一个隐含的最高位 0
 均匀分布
- 特殊值 —— $\text{exp} = 11\dots 11$
 - $\text{frac} = 000\dots 0$: 表示值 ∞ (无穷大) 或 **运算结果的溢出**
- $\text{frac} \neq 000\dots 0$: Not-a-Number (NaN), 表示无法确定数值的情况



4.5 非数值信息的表示

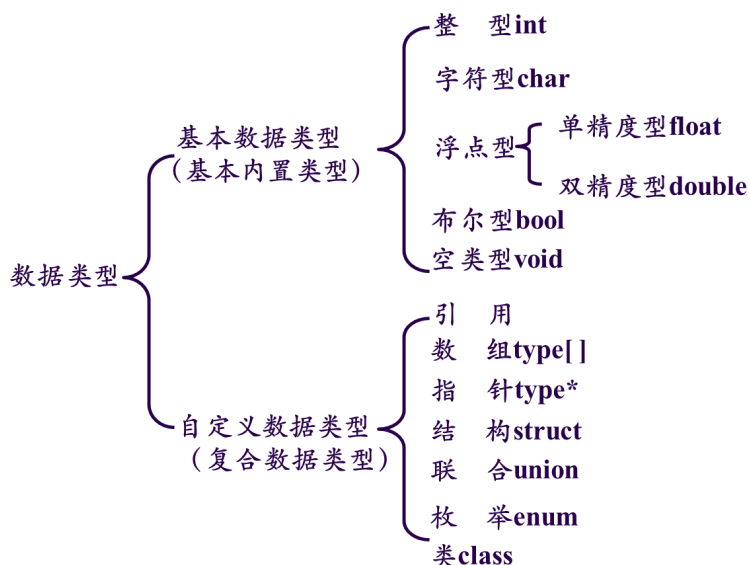
- ASCII码: 用7位二进制数编码字符, 能表示128个字符;
- EBCDIC码: 用8位二进制数编码字符, 能表示256个字符;
- GB2312-80国标码: 用两个7位二进制数编码一个汉字;

第二章：简单程序设计

1 C++语言概述

- 标准C++引入的可以由程序员命名的作用域，每个命名空间中可以放入一些相关的实体，例如 变量、函数、对象、类，以解决同名冲突问题，std: C++标准类库的一个名字空间。using指令：例如using namespace std，作用是引入std中的实体名到该程序；
- 词法记号：
 - 标识符：字母、数字、下划线构成，数字不能开头，区分大小写，不和关键词同名；
 - 关键字（也称为“保留字”）：都是小写的；
 - 文字：数字、字符、字符串、布尔文字；
 - 操作符（运算符）：与或非等；
 - 分隔符：小括号、中括号、大括号、逗号、分号等等；
 - 空白；

2 基本类型和表达式



2.1 基本数据类型

类型名	长度 (字节)	取值范围
bool	1	false, true
char	1	-128~127
signed char	1	-128~127
unsigned char	1	0~255
short (signed short)	2	-32768~32767
unsigned short	2	0~65535
int (signed int)	4 (或2)	-231~231-1
unsigned int	4 (或8)	0~232-1
long (signed long)	4 (或8)	-231~231-1
unsigned long	4 (或8)	0~232-1
long long	8	-263~263-1
unsigned long long	8	0~264-1
float	4	绝对值范围 3.4×10^{-38} 至 3.4×10^{38}
double	8	绝对值范围 1.7×10^{-308} 至 1.7×10^{308}
long double	8	绝对值范围 1.7×10^{-308} 至 1.7×10^{308}

2.2 常量

- 整型常量：十进制、八进制（如067, -067）、十六进制（如0xAB, -0xAB），存储时按照最小兼容类型算，可以加上常量后缀（L表示long，U表示unsigned）；
- 浮点型常量：小数表示法（如：-99.0, .456）、科学表示法（如：0.123E12, 67e-2），没有常量后缀（L表示long，F表示float）。
- 字符型常量：一对单引号中的单个字符。

\b 退格	\t 水平制表	\v 垂直制表	\a 报警
\f 换页	\" 双引号	\? 问号	
\n 换行	\' 单引号	\N 8进制常量	
\r 回车	\\ 反斜线	\xN 16进制常量	

- 字符串常量：一对双引号中的一系列字符。**空字符\0要注意！**
- 布尔常量：true, false。

2.3 变量

- 变量的作用范围；
- 变量的定义（有的时候顺便初始化）、赋值操作，区分初始化和赋值；
 - `int units_sold = 0;` //我们最熟悉的初始化形式

- `int units_sold = {0};` //列表初始化形式，花括号会warning“变窄”的操作
 - 区分`int units_sold[1] = {0}`，这是定义数组的
 - `int units_sold {0};` //第二种列表初始化形式，花括号会warning“变窄”的操作
 - `int units_sold (0);` //第四种初始化形式
 - 定义在函数之外的，默认初值为0；
 - 定义在函数之内 的，则不被初始化。
- 限定词：
 - `const`：符号常量，可以理解为对`#define`的优化；
 - `constexpr`：可以在编译时被求值，前提是它的所有参数都是编译时常量。如果用运行时变量调用它，它就表现得像一个普通函数，在运行时求值。

```
constexpr int s1 = square(5); // s1 在编译时就是 25
```

- `volatile`：用于编译系统会自动优化，告知编译器这个变量经常变化
- 变量的存储类别：
 - 局部变量，默认局部动态生存期；
 - `register`：最新版本废除，他建议编译器把变量存储在寄存器中，因此不能取址；
 - `extern`：全局变量默认。说明需要使用在程序的其他地方具有外部链接的对象；
 - `static`：静态全局变量，设置为内部链接（本文件可访问）。静态局部变量，具有静态生存期，函数内使用。
 - `mutable`：只能用于类数据成员，并且不能与`const`或者`static`同时使用，不能用来修饰引用变量。`mutable`关键字是为了突破`const`关键字的限制,被`mutable`关键字修饰的成员变量永远处于可变的状态，即使是在被`const`修饰的成员函数中。

2.4 运算符和表达式

优先级	运算符	结合性
1	()、[]、.、->、后置++、后置--	左
2	前置++、前置--、sizeof、&、*、+（正号）、-（负号）、~、!	右
3	（强制类型转换）	右
4	.*、->*	左
5	*、%、\、/	左
6	+、-	左
7	<<、>>	左
8	<、>、<=、>=	左
9	=、!=	左
10	&	左
11	^	左
12		左
13	&&	左
14		左
15	?:	右
16	=、*=、/=、%=、+=、-=、<<=、>>=、&=、^=、 =	右
17	,	左

2.4.1 算术运算符

- %运算的符号只取决于第一个运算数的符号。
- 除法运算如果是两个整数，其结果也是整数且“靠近零”，如：-5/2 = -2；
- 区分x++，++x；

2.4.2 赋值运算符

- 整型类别数据之间的转换：
 - 短到长：无符号数，高位补0；有符号数，高位补符号位。

```
unsigned char c;  
short int x;  
c = '\376';      /*(八进制数376) */  
x = c;           //11111110 -> 0000000011111110
```

- 长到短：截取低位赋值，丢弃高位。

```
unsigned char c;  
short int x;  
x = 0xff76;  
c = x;          //11111111 01110110 -> 01110110
```


2.4.3 逗号运算符

- `Exp1, Exp2, ..., Expn`: 表达式以最后一项`Expn`的值作为结果。

2.4.4 关系和逻辑运算符

- **短路原则:**

- `int m=0,n=0,a=0,b=0; (m=a>b)&&(n=a>=b): m = 0, n = 0`
- `int m=0,n=0,a=0,b=0; (m=a>=b)|| (n=a>=b): m = 1, n = 0`

2.4.5 条件运算符

- `Exp1 ? Exp2 : Exp3`

2.4.6 编译时运算符

- `sizeof(变量、数据类型名)`; 类型名必须加括号;
- 返回字节数;

2.4.7 位运算符

- 移位是有符号的右移;

2.4.8 混合运算时数据类型的转换

- 在算术运算和关系运算中如果参与运算的操作数类型不一致，编译系统会自动对数据进行转换（即**隐含转换**），基本原则是将低类型数据转换为高类型数据。赋值运算要求左值与右值的类型相同，若类型不同，编译系统会自动将右值转换为左值的类型。
- **显式转换**: `int(z)`, `(int)z`, `static_cast<int>(z)` 三种完全等价

3 数据的输入与输出

3.1 I/O流

- 将数据从一个对象到另一个对象的流动抽象为“流”。流在使用前要被建立，使用后要被删除。

3.2 `cout`, `cin`特殊用法

```
int num = 30;

cout << "十进制: " << dec << num << endl; // 输出: 十进制: 30
```

```

cout << "十六进制: " << hex << num << endl; // 输出: 十六进制: 1e
cout << "八进制: " << oct << num << endl;    // 输出: 八进制: 36

// 注意: 设置后会影响后续所有整数输出
cout << "还是八进制: " << num << endl;      // 输出: 还是八进制: 36

char a, b;
cout << "请输入两个字符 (例如: a b) : ";
cin >> ws >> a >> ws >> b; // 使用ws跳过输入前的任何空白
cout << "你输入的是: " << a << " 和 " << b << endl;

cout << "Hello" << ends << "World" << endl;

cout << setw(5) << setprecision(3) << 3.1415 << endl; // 3.14前面有一个空格

```

4 算法的基本控制结构

- if语句: 为了避免在嵌套的条件语句if-else中产生二义性, C++ 语言规定: else 子句总是与其之前最近的if 配对;
- switch语句:

```

switch(expression){    //字符型、枚举类型或整型表达式
    case constant1:      //常量表达式
        statement sequence
        break;
    case constant2:
        statement sequence
        break;
    .....
    default:
        statement sequence
}

```

– 允许的范式:

```

case 1:
case 2: cout<<"case1和2"<<endl;

```

- while语句、do-while语句;
- for语句: 注意浮点数存储的有限性, 可能需要引入little_bias;

```
for( auto &r : v )    //特殊用法
    r *= 2;
vector v = {0,1,2,3,4,5,6,7,8,9};
```

- break语句：用于终止离它最近的while、do while、for或switch;
- continue语句：用于while、do while和for;
- goto语句。

5 类型别名与类型推断

5.1 typedef声明

`typedef type name;` 特别地，typedef 没有创造新数据类型：

```
struct Point {
    int x;
    int y;
};
//调用
struct Point p1;
```

- 如果采用typedef:

```
typedef struct _Point {
    int x;
    int y;
} Point;
//调用
Point p1;
```

- 案例：

```
typedef    int    ARRAY[100]; //给我一个能装100个int的盒子类型，我管它叫
ARRAY
ARRAY    a,b,c;
//等价：    int a[100],b[100],c[100];
```

- 注意以下案例：

```
//const char * cstr=0; 和 (const char) * cstr=0;等价的，因此是指向常量的指针
```

```
typedef char *pstring;    //char* 这个类型，我给它起个外号叫 pstring,
typedef (char*) pstring
const pstring cstr = 0;   //const 修饰的是pstring，因此是常量指针。
const (char*) cstr = 0
//错误的理解: const char * cstr=0;    以为是指向常量字符的指针
//正确的理解: char* const cstr = 0;    一个常量指针，指向char
const pstring *ps;
// const pstring *ps; -> const (char*) *ps; -> (char* const) *ps;
// 指向char指针常量的指针
```

5.2 using声明

using 别名 = 类型名; 用法和前述类似。

5.3 auto编译器推断声明

auto i=0,j=1 编译器自动推断，发现问题则报错。

5.4 decltype声明

decltype(func()) valueB; 可以在不执行func()的时候得知其类型。

```
int x = 0;
decltype(x)  a;    // int
decltype((x)) d;    // int&! 注意：双重括号产生引用
```

第三章：函数

1 函数的定义与使用

1.1 函数定义

```
return-value-type  function-name(parameter list /*形式参数表*/)
{
    body of function    /*函数体*/
    return // C/C++中, 非void函数必须返回明确的值, 否则是未定义行为!
}
```

1.2 函数调用

```
function-name(parameter list /*实际参数表*/)
```

1.3 函数原型、声明

```
return-value-type  function-name(parameter list /*形式参数名称可以省略, 只需参数的类型*/ );
```

- 如果被调函数定义的位置放在主调函数之前, 用这种方法可以省去**原型说明**;
- 如果被调函数定义的位置放在主调函数之后, 必须有函数的原型说明。

1.4 嵌套和递归调用

- 明白栈的实现思路;
- 汉诺塔的实现代码:

```
void hanoi(int n, char src, char medium, char dest) {
    if (n == 1)
        move(src, dest);
    else {
        hanoi(n - 1, src, dest, medium);
        move(src, dest);
        hanoi(n - 1, medium, src, dest);
    }
}
```

1.5 参数传递

- 传入方式：值传递（按照地址传递是一种特例）、全局变量；
- 传出方式：返回值、引用传递、全局变量；

1.5.1 按值传递

- 形参和实参各有自己的内存空间；
- 形参的取值改变不影响实参。

1.5.2 按地址传递

- 形参和实参是指针类型；
- 形参和实参指向相同的内存空间。

```
void swap(int* x,int* y) {  
    definition;  
}  
int a=5,b=10;  
swap(&a,&b);           // 调用
```

- 引用(&)是标识符的别名：

```
int i, j;  
int &ri = i;  
j = 10;  
ri = j;//相当于 i = j;
```

1.5.3 引用传递

```
void reset(int &i){  
    definition;  
}  
reset(j);
```

1.5.4 可变数量形参

```
#include <iostream> // 用于标准输入输出, 如 cout  
#include <string>    // 用于 std::string 类型  
#include <initializer_list> // 引入 std::initializer_list 头文件  
  
// --- 示例一：用于日志记录或通用信息输出的函数 ---  
// 这个函数接受一个 string 类型的 initializer_list  
void log_info(std::initializer_list<std::string> messages) {  
    std::cout << "--- Log Info (Range-based for loop) ---" << std::endl;
```

```

// 使用范围-基于 for 循环遍历列表
for (const auto& msg : messages) { // msg 是 const std::string& 引用,
避免拷贝
    std::cout << msg << " ";
}
std::cout << std::endl << std::endl;
}

// --- 示例二: 用于打印错误或特定消息序列的函数 ---
// 这个函数也接受一个 string 类型的 initializer_list
void print_err(std::initializer_list<std::string> error_details) {
    std::cout << "--- Error Details (Iterator-based for loop) ---" <<
std::endl;
    // 使用传统的迭代器 for 循环遍历列表
    for (auto it = error_details.begin(); it != error_details.end();
++it) {
        std::cout << *it << " | "; // *it 解引用迭代器获取当前元素
    }
    std::cout << std::endl << std::endl;
}

int main() {
    // -----
    // 1. 调用 log_info 函数
    // 直接传入一个由 string 字面量组成的大括号列表
    log_info({"Hello", "Lambda", "C++11", "Features", "!"});

    // -----
    // 2. 调用 print_err 函数
    // 模拟一个错误检查场景, 混合使用 string 变量和字面量
    std::string expected_value = "Success";
    std::string actual_value = "Failure";

    if (expected_value != actual_value) {
        print_err({
            "Error",
            "Expected:", expected_value, // string 变量
            "Actual:", actual_value,    // string 变量
            "Details: Value mismatch!"
        });
    } else {
        print_err({"Status:", "All good!"});
    }

    // -----
    // 3. 传入空列表也是可以的
    std::cout << "--- Testing with empty list ---" << std::endl;
    log_info({}); // 传入一个空的 initializer_list

```

```

    print_err({}); // 传入一个空的 initializer_list
    std::cout << "--- End of empty list test ---" << std::endl <<
std::endl;

    // -----
    // 4. std::initializer_list 的其他用法 (例如用于容器初始化)
    //    虽然不是函数参数的直接用法, 但能帮助理解其理念
    std::vector<int> numbers = {10, 20, 30, 40, 50}; // 使用
initializer_list 初始化 std::vector
    std::cout << "Numbers in vector (initialized with initializer_list):
";
    for (int num : numbers) {
        std::cout << num << " ";
    }
    std::cout << std::endl;

    return 0;
}

```

2 constexpr函数

```

constexpr int len(int arg){return get_size()*arg;}
int arr[len(2)];
int i =2;
int a2[len(i)];

```

3 内联函数

```

inline return-value-type  function-name(parameter list)
{
    body of function    /*函数体*/
}

```

- 内联函数体内不能有循环语句和switch语句;
- 内联函数的声明必须出现在内联函数第一次被调用之前。
 - 对于一些函数体代码不是很大, 但又频繁地被调用的函数, 则引入内联函数。

- 在程序编译时，编译系统将程序中出现内联函数调用的地方用函数体进行替换。

4 带默认参数值的函数

- 有默认参数的形参必须在形参列表的最后；
- 如果一个函数有原型声明，且原型声明在定义之前，则默认参数值必须在函数原型声明中给出；

5 函数重载

- 形参个数不同：

```
int add(int x, int y);  
int add(int x, int y, int z);
```

- 形参类型不同：

```
int add(int x, int y);  
float add(float x, float y);
```

- 函数重载和默认参数同时使用，有可能引起二义性：

```
void DrawCircle(int r=0, int x=0, int y=0);  
void DrawCircle(int r);  
  
DrawCircle(20);
```

- 函数重载时，返回类型不在参数匹配检查范围之内。
- 函数调用时如果实参和形参类型不一致，编译器会自动做类型转换，但是函数重载时，这种类型转换有可能产生不能识别的错误。

```
void F_a(int x);  
void F_a(long int x);  
  
F_a(5.56);
```

6 使用C++系统函数

- 头文件以c开头: `cmath`, `cstdlib`, `cstdio`, `ctime`.....
- `double radian = angle * PI / 180;` 转化为弧度制;
- `sqrt`、`abs`等
- 随机数的实现:

```
#include <cstdlib> // 用于 rand() 和 srand()
#include <ctime>    // 用于 time()

int main() {
    // 1. 设置种子 (通常只设置一次, 在程序开始时)
    // 使用当前时间作为种子, 确保每次运行程序时生成不同的随机数序列
    std::srand(static_cast<unsigned int>(std::time(NULL)));

    // 2. 生成并打印几个随机数
    std::cout << "随机数 1: " << std::rand() << std::endl;
    std::cout << "随机数 2: " << std::rand() << std::endl;
    std::cout << "随机数 3: " << std::rand() << std::endl;

    // 3. 生成指定范围内的随机数 (例如: 1 到 100 之间)
    std::cout << "\n1 到 100 之间的随机数: " << std::endl;
    for (int i = 0; i < 5; ++i) {
        // (rand() % range) + min_value
        // rand() % 100 将生成 0 到 99 之间的数
        // + 1 使得范围变为 1 到 100
        int random_in_range = (std::rand() % 100) + 1;
        std::cout << random_in_range << " ";
    }
    std::cout << std::endl;

    // 4. 生成指定范围 [min, max] 的随机数
    // 例如: 生成 50 到 100 之间的随机数
    int min_val = 50;
    int max_val = 100;
    int range_size = max_val - min_val + 1;
    std::cout << "\n50 到 100 之间的随机数: " << std::endl;
    std::cout << (std::rand() % range_size) + min_val << std::endl;

    return 0;
}
```

第四章：类

1 面向对象程序设计的基本特点

1.1 结构体

- 结构体的缺省访问权限是public;
- 结构体格式:

```
struct 结构体名称 {  
    公有成员  
protected:  
    保护型成员  
private:  
    私有成员  
};  
struct Student {  
    int num;  
    string name;  
};  
Student stu = { 97001, "Lin Lin" }; // 初始化  
// 用stu.name调用;
```

1.2 抽象

- 数据抽象：描述某类对象的属性或状态;
- 代码抽象：描述某类对象的共有的行为特征;
- 抽象的实现：通过类的声明。

1.3 封装

- 将抽象出的数据成员、代码成员相结合，将它们构成一个整体。

1.4 继承

- 是C++中支持层次分类的一种机制，允许程序员在保持原有类特性的基础上，进行更具体的说明。

1.5 多态

- 同一名称，不同的功能实现方式。
- C++有强制多态（通过将一种类型的数据转换为另一种类型的数据来实现）；
- 重载多态（同一个名字赋予不同含义）；
- 类型参数化多态（模板）；
- 包含多态（虚函数）。

2 类和对象

- 类是具有相同属性和行为的一组对象的集合；

```
class Date                                //定义日期类Date
{
    public:                                //声明类成员.也称为类的说明部分
        void SetDate(int y = 0, int m = 0, int d = 0);
        void ShowDate( ){
            cout << year << month << day << endl;    // 类内定义的函数默认内
联函数
        };
    private:                               //private可以默认
        int year;
        int month;
        int day;
};

void Date::SetDate(int y, int m, int d) {    // void前面加上inline也可以成为
内联函数
    year = y;
    month = m;
    day = d;
}
```

- 内联函数的标志“inline”只代表程序员的建议，不代表编译器的决定！

2.1 访问控制权限

- private私有：只允许本类中的函数及友元访问，而其他的函数不能访问；
- protected保护：能被本类成员函数、派生类成员函数和友元访问，其他函数无法访问。

2.2 对象的赋值语句a = b

- 同类的对象之间可以赋值;
- 上面例子中的对象赋值时通过默认赋值运算符函数进行对象赋值。
- 当类中存在指针时，使用默认赋值运算符函数进行对象赋值，可能会产生错误，这个问题需要用赋值运算符重载解决。

3 构造函数和析构函数

3.1 构造函数

- **构造函数是成员函数**，函数体可写在类体内，也可写在类体外。
- 构造函数的函数名与类名相同，**且不指定返回值类型，它有隐含的返回值**，该值由编译系统内部使用。
- 构造函数可以没有参数，也可以有参数，因此可以重载，即可以定义参数不同的多个构造函数。
- 每个类都必须有一个构造函数。如果类中没有显式定义构造函数，则编译系统自动生成一个缺省形式的构造函数，作为该类的公有成员。一旦程序定义了自己的构造函数，则系统不再提供默认构造函数。此时，如果程序中没有定义一个无参数的构造函数，但又声明了一个没有初始化的对象，则由于系统不再提供默认构造函数而造成编译错误。
- 在定义对象时编译系统自动调用构造函数。
- 如果显式定义了构造函数，则默认构造函数将不存在。

⚠ 注意

1. **const变量**必须通过构造函数的**成员初始化列表**初始化;

```
const int id;
public:
    Person(int i){
        id = i;
    } //error
```

2. **成员初始化顺序与它们在类定义中的出现顺序一致**，而不是在初始值中出现的顺序!

```

class GoodExample {
public:
    int divisor; // 先声明 divisor
    int num;      // 后声明 num

    // 构造函数
    // 声明顺序和初始化列表顺序一致: divisor -> num, 若反过来也是正确的。
    GoodExample(int d, int n) : divisor(d), num(n / divisor) {
    // OK
        // 当 num(n / divisor) 执行时, divisor 已经被 d 初始化了
        std::cout << "Constructor body: num = " << num << ",
divisor = " << divisor << std::endl;
    }
};

```

3.1.1 默认构造函数（可以被复写）

1. 当不定义构造函数时，编译器自动产生默认构造函数。

- **解释：** 如果您的类中没有显式地定义任何构造函数（无论是带参数的还是不带参数的），C++ 编译器会为您自动生成一个公共的、无参数的默认构造函数。这个编译器生成的默认构造函数会负责调用其成员变量的默认构造函数（如果是类类型成员）或执行默认初始化（如果是基本类型成员）。

2. 在类中可以自定义无参数的构造函数，也是默认构造函数。

- **解释：** 您可以显式地定义一个不接受任何参数的构造函数。例如 `MyClass::MyClass() {}`。这取代了编译器自动生成的版本。

3. 全部参数都有默认形参值的构造函数也是默认构造函数。

- **解释：** 如果您定义了一个构造函数，并且它的所有参数都提供了默认值，那么这个构造函数也可以在不提供任何参数的情况下被调用。例如 `MyClass::MyClass(int x = 0, std::string s = "") {}`。

3.1.2 委托构造函数

- 可以理解为了重载：

```

Clock (int newH,int newN,int newS){
    hour=newH;
    minute=newN;
    second=newS;
}
Clock():Clock(0,0,0){} // 可以复用更复杂的clock初始化函数

```

3.1.3 复制构造函数

- 实现上如下:

```
class Point {    //Point 类的定义
public:
    Point(int xx=0, int yy=0) { x = xx; y = yy; }    //构造函数, 内联
    Point(const Point& p); //复制构造函数
    void setX(int xx) {x=xx;}
    void setY(int yy) {y=yy;}
    int getX() const { return x; } //常函数 (第5章)
    int getY() const { return y; } //常函数 (第5章)
private:
    int x, y; //私有数据
};

//成员函数的实现
Point::Point (const Point& p) {
    x = p.x;
    y = p.y;
    cout << "Calling the copy constructor " << endl;
}
//形参为Point类对象的函数
void fun1(Point p) {
    cout << p.getX() << endl;
}
//返回值为Point类对象的函数
Point fun2() {
    Point a(1, 2);
    return a;
}

//主程序
int main() {
    Point a(4, 5); //第一个对象a
    //情况一, 用a初始化b。第一次调用复制构造函数
    //Point b; b = a; (重载赋值运算符)!!!
    Point b = a;
    cout << b.getX() << endl;
    //情况二, 对象b作为fun1的实参。第二次调用复制构造函数, 引用则不涉及
    fun1(b);
    //情况三, 函数的返回值是类对象, 函数返回时调用复制构造函数
    b = fun2();
    //在情况二和情况三中, 编译器可能发生优化, 避免创建对象, 再执行复制构造函数。前者在运行栈的传参区域创建临时的对象, 后者称之为返回值优化RVO。
    cout << b.getX() << endl;
    return 0;
}
```

```

/*
具名返回值优化 (NRVO: Named Return Value Optimization)
针对场景: return local_named_object;
例如 return temp;。编译器可以将被返回的局部对象 temp 直接构造到调用者预留
的存储空间中 (例如 main 函数中 b 的位置), 而不需要先在 fun2 内部构造
temp, 然后将其复制到临时返回对象, 再将临时返回对象赋值给 b。
效果:
fun2 内部的 temp 对象不会被构造。
temp 到临时返回值的复制构造被省略。
如果同时省略了临时对象, 那么后续的赋值运算符也可能被省略, 直接导致 b 从一开
始就被构造为 fun2 的结果, 而不是经过赋值。
返回值优化 (RVO: Return Value Optimization) / 临时对象作为右值传递时的省
略
针对场景: return expression; 其中 expression 是一个纯右值 (prvalue), 例
如 return Point(30, 40);。
或者当函数参数是一个临时对象, 例如 fun1(Point(10, 20));。
编译器可以将被返回或被传递的临时对象直接构造到其最终目的地, 避免创建不必要的
临时副本。
*/

```

⚠ 注意

```

//尽量使用引用传递
void fun1(const Point &p){
    cout << p.getX() << endl;
    p.setX(1); //error, 没有声明const常函数!!
    //只有 const 成员函数才能被 const 对象 (或 const 引用/指针) 调用。
}

```

- 如果程序员没有为类声明复制构造函数, 则编译器自己生成一个隐含的复制构造函数。这个构造函数执行的功能是: 用作为初始值的对象的每个数据成员的值, 初始化将要建立的对象的对数据成员。

💡 概念

- 引入“explicit”的作用是避免隐式构造函数定义类型转换。比如正常是 `Line(Point(1),Point(4)).getLen()`, 如果在Point的构造函数前面没有explicit关键词, 运行 `Line(1,4).getLen()` 也会实现同样的效果, 这是不安全的。

3.1.4 移动构造函数

- 左值与右值：
 - 左值就是那些可以被赋值的东西，有名字，有内存地址，可以长期存在。
 - 右值就是那些零时产生、没有名字、不能被赋值、用完就丢的东西。
 - 左值引用 (&) —— 给“箱子”取个别名： `int &alias_x = x;`
 - 右值引用 (&&) —— 抓住临时的“物品”并拿来“废物利用”： `int &&temp_value = 10 + 20;`
 - `std::move()`：是一个类型转换，它把一个左值强制转换为右值引用。它告诉编译器：“我承诺这个左值我以后不再用它的旧内容了，你可以安全地从它那里‘移动’资源。”
- 移动构造函数的写法：

```
MyString(MyString&& other) noexcept : data(other.data),
size(other.size) {
    other.data = nullptr; // 将源对象的指针置空
    other.size = 0;       // 将源对象的大小置0
    std::cout << "Move constructor from '" << (data ? data :
"nullptr") << "'" << std::endl;
}

MyString& operator=(MyString&& other) noexcept {
    if (this != &other) {
        delete[] data; // 释放旧资源
        data = other.data; // 转移资源
        size = other.size;
        other.data = nullptr; // 源对象置空
        other.size = 0;       // 源对象置0
    }
    return *this;
}
```

- 理解 `MyString s1 = createString("Hello");`
 - 首选：RVO;
 - 次选：移动构造函数 (如果存在且可行如果存在且可行)
 - 最后选择：拷贝构造函数 (如果没有移动构造函数如果没有移动构造函数)
- `default`、`delete`函数：

```

MyStr() = default;           // 显式要求编译器生成默认无参构造函数

MyStr(const MyStr &str) = default;    // 生成默认拷贝构造函数
MyStr(MyStr &&str) = default;         // 生成默认移动构造函数
~MyStr() = default;                 // 生成默认析构函数
NonCopyable() = default;
NonCopyable(const NonCopyable&) = delete;           // 禁止拷贝构造
NonCopyable& operator=(const NonCopyable&) = delete; // 禁止拷贝赋值

```

3.2 析构函数

- 析构函数也是类的一个公有成员函数，它的名称是由类名前面加“~”构成，也不指定返回值类型。
- 和构造函数不同的是，析构函数不能有参数，因此不能重载。
- 不能是const

```

class Point {
public:
    Point(int xx,int yy);
    ~Point();
    //...其他函数原型
private:
    int x, y;
};
Point::Point(int xx,int yy) {
    x = xx;
    y = yy;
}
Point::~~Point() {
}

```

3.3 完整案例

```

#include <iostream>
#include <cstring> // For strlen, strcpy

using namespace std;

class MyString{
private:
    char *str; // 动态分配的 char 数组，存储字符串

```

```

public:
    // 1. 默认构造函数
    MyString() : str(NULL) { // 使用初始化列表, 将指针初始化为 NULL
        cout << "Default constructor called, str is NULL. obj addr: " <<
(void*)this << endl;
    }

    // 2. 参数化构造函数
    // 接受 const char*, 更加通用和安全, 因为函数不修改传入字符串
    MyString(const char *s) : str(NULL) { // or const char* const
&s, 但没必要引用传递
        cout << "Parameterized constructor called, obj addr: " <<
(void*)this << endl;
        if (s) { // 检查传入指针是否为空
            str = new char[strlen(s) + 1]; // 动态分配内存
            strcpy(str, s); // 复制内容
            cout << " Copied: " << str << ", internal addr: " <<
(void*)str << endl;
        } else {
            cout << " Input string is NULL." << endl;
        }
    }

    // 3. 复制构造函数 (深拷贝)
    MyString(const MyString &other) : str(NULL) {
        cout << "Copy constructor called, obj addr: " << (void*)this << "
from " << (void*)&other << endl;
        if (other.str) {
            str = new char[strlen(other.str) + 1];
            strcpy(str, other.str);
            cout << " Deep copied: " << str << ", internal addr: " <<
(void*)str << endl;
        } else {
            cout << " Source object has NULL string." << endl;
        }
    }

    // 4. 析构函数 (释放内存)
    ~MyString() {
        if (str) {
            cout << "Destroying: " << str << ", internal addr: " <<
(void*)str << ", obj addr: " << (void*)this << endl;
            delete[] str;
            str = NULL;
        } else {
            cout << "Destroying (NULL string): obj addr: " << (void*)this
<< endl;

```

```

    }
}

// 5. 赋值运算符重载 (深拷贝, 处理自我赋值, 防止内存泄漏和双重释放)
MyString& operator=(const MyString &other) {
    cout << "Assignment operator called, obj addr: " << (void*)this
<< " from " << (void*)&other << endl;
    if (this == &other) { // 1. 自我赋值检查
        cout << " Self-assignment detected, returning early." <<
endl;
        return *this;
    }

    // 2. 释放当前对象旧的内存
    if (str) {
        cout << " Releasing old memory: " << str << ", internal
addr: " << (void*)str << endl;
        delete[] str;
        str = NULL; // 提前置空, 安全起见
    }

    // 3. 复制 other 的内容到新分配的内存
    if (other.str) {
        str = new char[strlen(other.str) + 1];
        strcpy(str, other.str);
        cout << " Copied new string: " << str << ", internal addr: "
<< (void*)str << endl;
    } else {
        str = NULL; // 如果源对象是空字符串, 则当前对象也为空
        cout << " Source object has NULL string, current string set
to NULL." << endl;
    }
    return *this; // 4. 返回当前对象的引用
}

// 6. 设置字符串的方法
void setStr(const char* s) {
    cout << "setStr called on obj addr: " << (void*)this << endl;
    if (str) {
        cout << " Releasing old memory: " << str << ", internal
addr: " << (void*)str << endl;
        delete[] str;
        str = NULL;
    }

    if (s) {
        str = new char[strlen(s) + 1];
        strcpy(str, s);
    }
}

```

```

        cout << "  New string set: " << str << ", internal addr: " <<
(void*)str << endl;
    } else {
        cout << "  Input string is NULL, internal string set to
NULL." << endl;
        str = NULL;
    }
}

// 7. 打印字符串的方法
void printStr() const {
    if (str) {
        cout << "MyString (obj addr: " << (void*)this << "): " << str
<< ", internal addr:" << (void*)str << endl;
    } else {
        cout << "MyString (obj addr: " << (void*)this << "): (null),
internal addr:0" << endl;
    }
}

// 8. 改变字符串内容的方法
void changeStr(const char* s){
    cout << "changeStr called on obj addr: " << (void*)this << endl;
    if (s == NULL) {
        if (str) {
            cout << "  Clearing current string: " << str << ",
internal addr: " << (void*)str << endl;
            delete[] str;
            str = NULL;
        } else {
            cout << "  Current string already nullptr, input is
nullptr." << endl;
        }
        return;
    }

    if (str == NULL || strlen(s) > strlen(str)){
        if (str) { // 先释放旧内存 (因为它不够长或为空)
            cout << "  Reallocating memory (old too small/null) for:
" << s << ", old addr: " << (void*)str << endl;
            delete[] str;
        } else {
            cout << "  Allocating new memory for: " << s << endl;
        }
        str = new char[strlen(s) + 1];
    }

    strcpy(str, s);
}

```

```

        cout << " String changed to: " << str << ", new internal addr: "
<< (void*)str << endl;
    }
};

int main(){
    // 修正 `a` 的定义, 使其成为 `const char*`, 符合现代C++处理字符串字面量的
    习惯
    const char* a = "haha"; // 推荐: 字符串字面量通常是 const char*

    cout << "--- Creating str1 ---" << endl;
    MyString str1(a); // 使用参数化构造函数
    str1.printStr();

    cout << "\n--- Creating str2 ---" << endl;
    MyString str2; // 使用默认构造函数, str 为 nullptr
    str2.printStr();

    cout << "\n--- Setting str2 ---" << endl;
    str2.setStr("hello world"); // 使用 setStr 方法
    str2.printStr();

    cout << "\n--- Creating str3 and assigning str1 to it ---" << endl;
    MyString str3; // 默认构造
    str3 = str1; // 赋值运算符
    str3.printStr();

    cout << "\n--- Creating str4 and assigning str1 to it ---" << endl;
    MyString str4 = str1; // 复制构造函数
    str4.printStr();

    cout << "\n--- Modifying str1 ---" << endl;
    str1.changeStr("new value for str1");
    str1.printStr(); // str1 改变了
    str3.printStr(); // str3 不受影响 (深拷贝证明)

    cout << "\n--- Str2 changing string to a shorter one ---" << endl;
    str2.changeStr("hi"); // 改变为更短的字符串
    str2.printStr();

    cout << "\n--- Str3 setting null string ---" << endl;
    str3.setStr(NULL);
    str3.printStr();

    cout << "\n--- End of main - Destructors will be called ---" << endl;
    return 0; // 程序结束时, str3, str2, str1 会按创建顺序的逆序调用析构函数
}

```

4 类的组合

4.1 构造时的处理方式

- 成员对象构造函数调用顺序：按对象成员的声明顺序，先声明者先构造。
 - `Line::Line(Point xp1, Point xp2) : p1(xp1), p2(xp2)`，这是对p1和p2进行了复制构造函数的创建；
 - p1和p2哪个先初始化和声明顺序相关。
- 初始化列表中未出现的成员对象，调用用默认构造函数（即无形参的）初始化。
 - 调用相应的默认构造函数，如果没有，则报错！

4.2 前向引用声明

- 如果需要在某个类的声明之前，引用该类，则应进行前向引用声明。主要用于循环调用。
- 尽管使用了前向引用声明，但是在提供一个完整的类声明之前，不能声明该类的对象。只能用于定义指针、引用、以及用于函数形参的指针和引用。
- **需要明确：类的定义可以只包括成员函数的声明。类的声明是简单的class A;这种。**
- 完整案例：
 - company.h

```
#ifndef COMPANY_H
#define COMPANY_H

class Employee;

class Company {
public:
    Company(int id, int ceoid);
    ~Company();
    void addEmployee(Employee &emp);
    void getceo();

    Employee* ceo;
    int id;
};

#endif
```

- company.cpp

```

#include <iostream>
#include "company.h"
#include "employee.h"
using namespace std;

Company::Company(int id, int ceoid):id(id){
    ceo = new Employee(ceoid);
    ceo->assignCompany(*this);
}

Company::~~Company(){
    delete ceo;
    ceo = NULL;
}

void Company::addEmployee(Employee &emp){
    cout << "Adding id:" << emp.id << endl;
}

void Company::getceo(){
    cout << "ceo-id:" << ceo->id << endl;
}

```

– employee.h

```

#ifndef EMPLOYEE_H
#define EMPLOYEE_H

class Company;

class Employee{
public:
    Employee(int id);
    ~Employee();
    void assignCompany(Company &comp);

    Company* current;
    int id;
};

#endif

```

– employee.cpp


```

#include <iostream>
#include "employee.h"
#include "company.h"
using namespace std;

Employee::Employee(int id):id(id), current(NULL){}

Employee::~Employee(){}

void Employee::assignCompany(Company &comp){
    current = &comp;
    comp.addEmployee(*this);
}

```

– main.cpp

```

#include <iostream>
#include "employee.h"
#include "company.h"
using namespace std;

int main(){
    Company inno(1627, 678);
    inno.getceo();

    Employee zhl(727);
    zhl.assignCompany(inno);

    return 0;
}

```

– Makefile:

```

my_program: main.o company.o employee.o
    g++ main.o company.o employee.o -o myprogram

main.o: main.cpp
    g++ -c main.cpp -o main.o

company.o: company.cpp
    g++ -c company.cpp -o company.o

employee.o: employee.cpp
    g++ -c employee.cpp -o employee.o

clean:

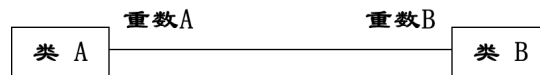
```

```
rm -f my_program *.o
```

- 编译“make”，删除用“clean”。

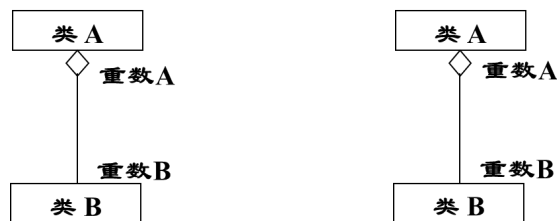
5 UML图形标识

- UML (Unified Modeling Language) 语言是一种可视化的面向对象建模语言。这里主要讨论类图
- **关联 (Association)**：不同对象实例间的一种结构连接。是最常见、最弱的一种关系。

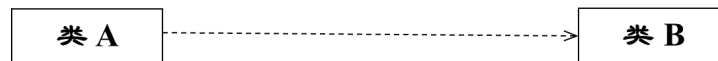


- 重数A是每个B对象和多少个A对象发生作用。

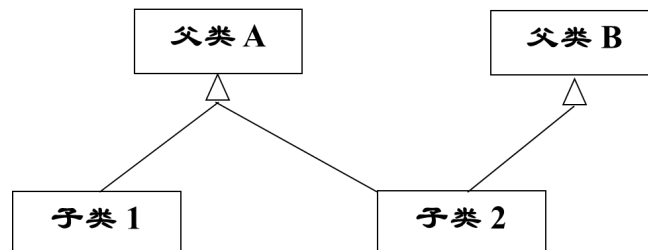
- **聚合 (Aggregation)**：一种特殊的关联，表示“整体-部分”关系，但部分的生命周期可以独立于整体。
- **组合 (Composition)**：一种更强的“整体-部分”关系，部分与整体的生命周期强相关，整体不存在，部分也就不存在。（菱形涂黑）



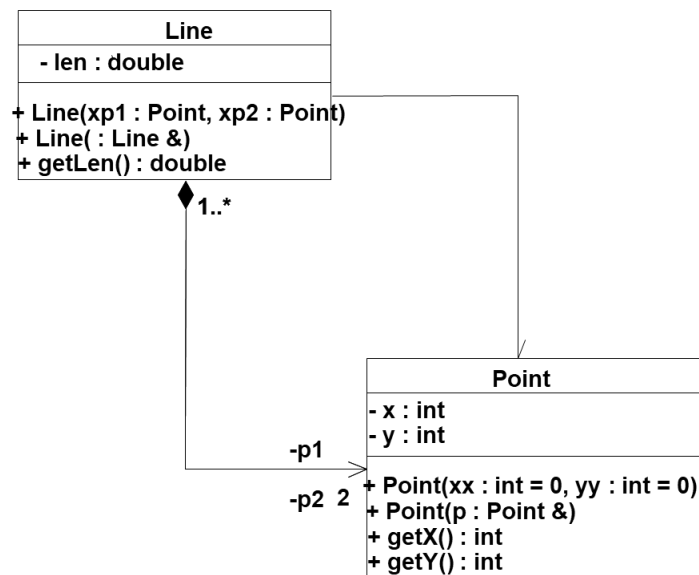
- **依赖 (Dependency)**：一个类的改变可能会影响另一个类。



- **泛化 (Generalization)**：即继承，表示“is-a”关系。



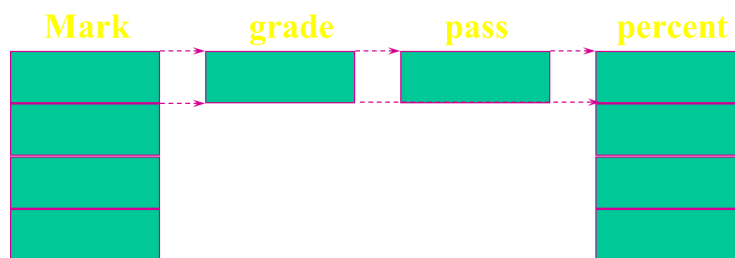
- **实现 (Realization)**：接口与实现类之间的关系。
- 案例：



6 联合体

```

union 联合体名称 {
    公有成员
protected:
    保护型成员
private:
    私有成员
};
  
```



- 这样会得到错误的i值:

```

union {
    int i;
    float f;
};
//在程序中可以这样使用:
i = 10;
f = 2.2;
cout << i << endl;
  
```

- 使用举例，前者是匿名，后者是有名:

```

#include <iostream>
using namespace std;

enum type {INT, FLOAT, CHAR};

class DataType{
public:
    DataType(type t, int v) : t(t), v1(v){ }
    DataType(type t, float v) : t(t), v2(v){ }
    DataType(type t, char v) : t(t), v3(v){ }
    ~DataType(){}
    void add(const DataType &d){
        if (t != d.t){
            cout << "error" << endl;
            return;
        }else{
            if(t == 0) v1 += d.v1;
            else if (t == 1) v2 += d.v2;
            else if (t == 2) v3 += d.v3;
        }
    }
    void show(){
        cout << t << " ";
        cout << v1 << " ";
        cout << v2 << " ";
        cout << v3 << endl;
    }
private:
    type t;
    union {
        int v1;
        float v2;
        char v3;
    };
};

int main(){
    DataType d1(type::INT, 66);
    d1.show();
    DataType d2(type::FLOAT, 10.5f);
    d2.show();
    DataType d3(type::CHAR, '1');
    d3.show();
    DataType d4(type::FLOAT, 1.4f);
    d4.show();
}

```

```

        d1.add(d2);
        d2.add(d4);
        d2.show();

        return 0;
    }

// class DataType{
// public:
//     // 定义联合体类型
//     union value {
//         int v1;
//         float v2;
//         char v3;

//         // 联合体的构造函数
//         value(int i) : v1(i) {}
//         value(float f) : v2(f) {}
//         value(char c) : v3(c) {}
//     };

//     // 构造函数
//     DataType(type t, int v) : t(t), val(v) {}
//     DataType(type t, float v) : t(t), val(v) {}
//     DataType(type t, char v) : t(t), val(v) {}

//     ~DataType(){}

//     void add(const DataType &d){
//         if (t != d.t){
//             cout << "error" << endl;
//             return;
//         }else{
//             if(t == INT) val.v1 += d.val.v1;
//             else if (t == FLOAT) val.v2 += d.val.v2;
//             else if (t == CHAR) val.v3 += d.val.v3;
//         }
//     }

//     void show(){
//         cout << t << " ";
//         if(t == INT)
//             cout << val.v1 << endl;
//         else if (t == FLOAT)
//             cout << val.v2 << endl;
//         else if (t == CHAR)
//             cout << val.v3 << endl;
//     }

```

```
// private:
//     type t;
//     value val; // 命名的联合体成员变量
// };
```

7 枚举类型

```
enum W {SUN=7, MON=2, TUES, WED, THU, FRI, SAT } a; // enum W是类型名,
a是变量
a = TUES; //值为3, 依次递增, SAT和SUN同为7
cout << a << endl;
```

- 如果定义了类型W，定义变量的时候可以简单写为 `W a`，这是C++引入的语法糖。
- C++后来引入了枚举类，也称作限定作用域的枚举类：

```
#include<iostream>
using namespace std;
enum class Side{ Right, Left };
enum class Thing{ Wrong, Right }; //不冲突
int main()
{
    Side s = Side::Right;
    Thing w = Thing::Wrong;
    cout << (s == w) << endl; //编译错误，无法直接比较不同枚举类，同时可
    以理解输入Side::Right的值是会报错的
    return 0;
}
```

8 位域

- 在类或结构体定义中，位域的定义方式：数据类型说明符 成员名：位数；
 - 位数是二进制的占用。

```
#include <iostream>
using namespace std;

class CPU{
public:
```

```

    CPU(unsigned fre, unsigned bitlong, unsigned core, unsigned
thread)
:fre(fre),bitlong(bitlong),core(core),thread(thread){}
void info(){
    cout << "fre:" << fre << endl;
    cout << "bitlong:" << bitlong << endl;
    cout << "core:" << core << endl;
    cout << "thread:" << thread << endl;
}
private:
    unsigned fre : 12;
    unsigned bitlong : 7;
    unsigned core : 4;
    unsigned thread : 1;

};

int main(){
    CPU c1(1000, 32, 8, true);
    c1.info();
    CPU c2(1000, 32, 20, true); // overflow
    c2.info();

    return 0;
}

```

第五章：数据的共享与保护

1 标识符的作用域与可见性

- 作用域的含义：一个标识符的作用域指程序中可以访问（可见到）该标识符所代表的变量的区域。
- 对于两个嵌套的作用域，如果在内层作用域内声明了与外层作用域中同名的标识符，则外层作用域的标识符在内层不可见。

1.1 函数原型作用域

- 其作用域始于"("，结束于")"。这个区域说明的形参标识符，仅在此处可用，因此形参在原型的位置是属于可有可无的。

1.2 局部作用域(块作用域)

- 函数的形参，在块中声明的标识符，其作用域自声明处起，限于块中；

1.3 类作用域

- 类的成员具有类作用域，其范围包括类体和非内联成员函数的函数体。
- 如果在类作用域以外访问类的成员，要通过类名（访问静态成员），或者该类的对象名、对象引用、对象指针（访问非静态成员）。

1.4 文件作用域

- 作用域开始于声明点，结束于文件尾。
- PPT中类似于全局变量可以属于文件域，更恰当的应该是static的变量、

1.5 命名空间作用域

- **命名空间作用域** 指的是一个标识符（变量、函数等）在特定命名空间内可见和可访问的范围。
- **全局作用域** 可以被认为是“匿名”的、最顶层的命名空间。**所有在任何命名空间之外定义的标识符**都位于这个全局命名空间中。


```

namespace myown1
{
    string user_name="myown1";
}
//use
    myown1::username;    //域解析运算符，作用域运算符
//or
    using namespace myown1;
    username;
//or
    using myown1::username;    // 因为是变量，没有namespace关键词
    username;

```

- **匿名命名空间：**是一个需要显式声明的没有名字的命名空间。匿名命名空间常被用来屏蔽不希望暴露给其他源文件的标识符，因为每个源文件的匿名命名空间是不同的。

```

namespace
{
    //匿名命名空间内的各种声明（函数声明、类声明.....）
}

```

- 文件组织结构：exercise_namespace.cpp中定义了匿名namespace，同时有接口函数。main.cpp文件调用接口文件进行间接访问。

```

//exercise_namespace.cpp
#include <iostream>
using namespace std;
namespace {
    int banana = 3;
    int add(int x, int y){
        return x + y;
    }
}

namespace hasName{
    int apple = 1, tree = 2;
    int sub(int x, int y){
        return x - y;
    }
}

int using_add(int num){
    return add(num, banana);
}

```

```
//main.cpp
#include <iostream>
#include "exercise_namespace.cpp"
using namespace std;

int using_add(int);

int main(){
    cout << using_add(66) << endl;
    cout << hasName::sub(hasName::apple,hasName::tree) << endl;
    return 0;
}

//Makefile
program: main.cpp
    g++ main.cpp -o program

clean:
    rm -f program *.o
```

2 生存期（或称存储周期、存在时间）

- 标识符所代表的变量存在于内存中的时间；

2.1 静态生存期

- 这种生存期与程序的运行期相同。
- 在**文件作用域**中声明的对象（习惯称**全局变量**）具有这种生存期。
- 在**函数内部声明静态生存期对象**，要冠以关键字**static**。静态局部变量具有全局寿命，局部可见性。例如：static int i;
- 静态生存期的变量未被初始化时，会被**以0值初始化**。

2.2 动态生存期

- **块作用域中声明的**，没有用static修饰的对象是动态生存期的对象（习惯称局部生存期对象）。
- 开始于程序执行到声明点时，结束于命名该标识符的作用域结束处。
- 局部生存期变量不指定初值时，**初值将不确定**。

3 类的静态成员（和类关联）

- 解决同一个类的不同对象之间的数据和函数共享问题;

3.1 静态数据成员

- 必须在类外定义和初始化，用(::)来指明所属的类。
- 静态数据成员可以是public、private、protected权限之一。

3.2 静态函数成员

- 静态成员函数只能引用属于该类的静态数据成员或静态成员函数。
- 调用方式：

类名::静态成员函数名 (实参表) ;
对象.静态成员函数名 (实参表) ;

3.3 综合案例

Point
- x : int - y : int <u>- count : int = 0</u>
+ Point(xx : int = 0, yy : int = 0) + getX() : int + getY() : int + Point(p : Point &) <<static>>+ showCount() : void

```
#include <iostream>
using namespace std;
class Cat{
public:
    Cat(){
        numOfCats ++;
    }
    static int getNumOfCats(){
        return numOfCats;
    }
private:
    static int numOfCats;
```

```
};

int Cat::numOfCats = 0;

int main(){
    Cat c1,c2,c3;
    cout << Cat::getNumOfCats() << endl;
    cout << c1.getNumOfCats() << endl;
    return 0;
}
```

4 类的友元

- 友元是C++提供的一种破坏数据封装和数据隐藏的机制。**单向的**、注意逻辑“如果将类B声明为类A的友元，那么类B中的函数可以访问类A的全部成员”这句话是正确的。

4.1 友元函数

- 友元函数是在类声明中由关键字friend修饰说明的**非成员函数**，在它的函数体中能够通过对象名访问 private 和 protected成员。
- 友元函数**不是类的成员函数**，因此对友元函数指定访问权限无效，友元函数声明可以放在public、private、protected的**任意段**中

4.1.1 不属于别的类的友元函数

```
friend float dist(Point &, Point &);
```

4.1.2 属于别的类的友元函数

```
friend void M::Setab( N &);
```

4.2 友元类

- 若一个类为另一个类的友元，则此类的所有成员函数都能访问对方类的私有成员；**不具备传递性**。
- 在A类中，定义 friend class B，**意味着A说B是我的朋友**，B类可以访问A中的protected和private。

4.3 综合案例

```
#include <iostream>
using namespace std;

class X; // 前向声明, 两个类无法互相声明对方的某个特定成员函数 为友元。这在
C++语法和编译规则上是无法实现的。只能声明class作为友元。

class Y {
public:
    void g(X* x); // 只声明, 定义放在x之后
};

class Z {
public:
    void f(X* x); // 只声明, 定义放在x之后
};

class X {
public:
    X();
    friend void Y::g(X*); // 现在Y已经定义, 编译器知道g(X*)存在
    friend class Z;       // Z已经定义
    friend void h(X*);     // 普通函数友元
    void print_i() {
        cout << i << endl;
    }
private:
    int i;
};

X::X() : i(0) {}

// 现在可以定义Y和Z的成员函数, 因为X已经完整定义
void Y::g(X* x) {
    (x->i) += 1;
}

void Z::f(X* x) {
    x->i += 5;
}

void h(X *x) {
    x->i += 10;
}

int main() {
```

```

X x;
Y y;
Z z;
y.g(&x);
z.f(&x);
h(&x);
x.print_i(); // 输出: 16
return 0;
}

```

5 共享数据的保护

- 涉及到“**最小授权原则**”，即从安全性的角度，不允许相应的代码修改权限之外的东西。

①一般常量： `int const x=2;`或 `const int x=2;` `const` 变量**必须初始化**；

- 有一类**指向常量的指针**： `const int* p;`。

②**常数组**： `const int x[10] = {1, 2}`，**必须同时初始化**；

③**常指针**：指针不能指向别的地方，如 `int* const p;`，**必须定义时初始化**。结合指向常量的指针（不需要立即赋值），可以得到一种**复合类型** `const int* const p`，在函数形参中可以写成 `const char* const &s`，可以揣摩含义！

④**常引用**： `const` 类型说明符& 引用名；常引用所引用的对象不能被更新，如果是形参可以这么写，但是**在别名时必须同时绑定**，即 `const int& other_name = name`。

⑤**常对象**：类名 `const` 对象名 或者 `const` 类名 对象名，常对象在定义的时候必须被初始化，之后不允许再修改。

⑥**常成员函数**：类型说明符 函数名（参数表）`const`；**常对象的唯一接口！！**，通过常对象，只能调用它的常成员函数。不允许修改任何非mutable变量。

- 在线类中，`double getLen() const`是合理的，但是会更新len变量，这个的处理方式是给len增加mutable权限，这样永远处于可变状态下。

⑦**常数据成员**：如果在一个类中说明了常数据成员，那么任何函数中都不能对该成员赋值。构造函数对该成员进行初始化，就**只能通过初始化列表**。

- 存在 `static const int x;` 这种定义的可能，主要目的是减少存储进行共享，让语义更加清晰。

⑧常字符串: `const char *STRING1 = "program";` 这样语法才正确!

6 多文件结构和编译预处理命令

6.1 外部变量

- 文件作用域中定义的变量，缺省情况下都是外部变量，但在其它文件中如果需要使用这一变量，需要用`extern`关键字加以声明。

6.2 外部函数

- 类之外的函数都可以在不同的编译单元中被调用，只要在调用之前进行引用性声明（即声明函数原型）即可。也可以在声明函数原型或定义函数时用`extern`修饰，其效果与不加修饰的缺省状态是一样的。
- 在调用文件中声明`extern int add(int, int)`和`int add(int, int)`没有区别。

6.3 编译预处理指令

#include<文件名>：按标准方式搜索，文件位于C++系统目录的include子目录下

#include"文件名"：先在当前目录中搜索，若没有，再按标准方式搜索。

#define 宏定义指令：

- 定义符号常量（**不带参数的宏**），很多情况下已被`const`定义语句取代。
 - `#define WIDTH (80)`
 - `#define LENGTH (WIDTH+40)` // **注意加括号!**
- 定义**带参数宏**，已被内联函数取代。
 - `#define POWER(x) ((x)*(x))` // **注意加括号!**
 - **宏名和参数之间不能有空格。**

#undef：删除由`#define`定义的宏，使之不再起作用。

#if #elif #else #endif：条件编译指令。

#ifdef 标识符 #else #endif：条件编译指令。

#ifndef 标识符 #else #endif：条件编译指令。

第六章：数组、指针和字符串

1 数组

1.1 数组的声明与使用

- 类型说明符 数组名[常量表达式][常量表达式]..... ;
- 数组元素下标从0开始，地址名是地址常量，不是变量，因此a[100]中a++是语法错误的；
- 越界使用存储单元：C++编译系统不检查。

1.2 数组的存储和初始化

- 数组元素在内存中顺次存放，它们的地址是连续的，二维数组是“一维数组的一维数组”。
- 初始化方法：
 - 在声明数组时对数组元素赋以初值：`int a[3][4]={1,2,3,4,5,6,7,8,9,10,11,12};` 或者 `int a[3][4]={{1,2,3,4},{5,6,7,8},{9,10,11,12}};`
 - 可以只给一部分元素赋初值：`int a[3][4]={{1},{0,6},{0,0,11}};` 没有设置的元素默认被初始化为零。
 - 在对全部数组元素赋初值时，可以不指定数组长度：`int a[][4]={1,2,3,4,5,6,7,8,9,10,11,12};`

1.3 数组作为函数参数

- 数组名作参数，形、实参数都应是数组名。传送的是数组首地址。对形参数组的改变会直接影响到实参数组。
- 可以理解为**传值中的传地址**。
- 在声明函数的时候形如：`void func(int [])`。

1.4 特殊类型的数组

- 对象数组：`Point a[2]={Point(1,2),Point(3,4)};`
 - 初始化最好的方式，搞一个默认参数的构造函数。
 - 析构函数会按照构造的相反顺序执行。
- **指针数组**：意思是指针的数组，`int *p[10]` 或者 `int *(p[10])`，括号和星号是同优先级的，因此从右向左结合。
- **数组指针**：意思是数组的指针，如 `int (*p)[10]; int a[][10]; p = a;`


```

#include <iostream>
using namespace std;

int main() {
    int a[3][10] = {
        {1, 2, 3, 4, 5},           // 第一行, 后5个元素自动为0
        {10, 20, 30},             // 第二行, 后7个元素自动为0
        {100, 200, 300, 400, 500} // 第三行, 后5个元素自动为0
    };
    int (*p)[10];
    p = a;

    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 10; j++) {
            // 三种等价的访问方式:
            // 1. p[i][j] - 最直观
            // 2. (*(p + i) + j) - 指针运算方式
            // 3. (*(p + i))[j] - 混合方式
            cout << p[i][j] << " ";
        }
        cout << endl;
    }
    return 0;
}

```

① Note

以下不正确的定义语句和赋值语句是 **【正确答案: A C D】**

- A. `int b[3][5], (*p)[3], (*q)[5]; p = b; q = b;`
- B. `float b[3][5], * p[3]; p[0] = b[0]; p[2] = *b + 4;`
- C. `double b[3][5], s[5][3], * q; q = b; s = q;`
- D. `int b[10], * q; char* s; q = b; s = b;`

2 指针

2.1 概念

- 指针与指针变量;
- 直接访问与间接访问;
- 地址运算符&、指针运算符*;
- NULL: 技术角度看, 就是指向内存位置0的指针, 但是C++11使用nullptr关键字, 使表达更准确;
- 空类型指针: void*. 表示不指定p是指向哪一种类型数据的指针变量;

2.2 指针变量初始化

- 全局/静态指针变量未初始化，系统自动初始化为NULL，此时指向对象不存在；局部指针未初始化指向未知；
- 不要用一个内部 auto 变量去初始化 static 指针。

2.3 指针运算

- 指针加x，减x运算，指向下x个或前x个数据。
 - `y=px++` 相当于 `y=(px++)`, (和++优先级相同，自右向左运算)
- `p1 - p2` : `p1`和`p2`指向的对象之间间隔的数据个数；
- 大于、小于、等于、不等，这些关系运算。

2.4 指针函数与函数指针

- 指针函数：不要将非静态局部地址用作函数的返回值，可以：
 - 动态内存分配new出来的指针；
 - 全局开辟的指针；
 - 函数调用的时候传过来的指针；
- 函数指针；

```
#include <iostream>
using namespace std;

int func1 (int x){
    cout << x << endl;
    return 100;
}

void func2 (int x, int (*p)(int)) {
    cout << (*p)(x) << endl;
    cout << p(x) << endl;    //两种都OK
}

int main(){
    int (*p)(int) = func1;
    int (*p1)(int) = &func1;    //'&' 是可选的，效果一样

    func2(10, p);
    func2(20, p1);
    return 0;
}
```

2.5 对象指针

```
Point a(5,10);
Point *ptr;
ptr=&a;
//ptr->getx() 相当于 (*ptr).getx();
```

- this指针：是一个指向自己的**常量指针**，隐含于每一个类的成员函数中的特殊指针。在一般使用的时候**缺省**，在需要使用对象自己的是否可以用于返回。this指针作为隐含参数传递给非静态成员函数；在成员函数中返回对象本身通常使用 *this：
obj.add(5).multiply(2).add(3);
- 非静态成员指针：

```
// func
memberFuncPtr = &Calculator::add;    //&不能少!
(calc.*memberFuncPtr)(5);    或者    (calcPtr->*memberFuncPtr)(2);

//int
int Person::*id = &Person::id;        //&不能少!
person.*namePtr    或者    personPtr->*namePtr
```

- 静态成员指针：对类的静态成员的访问不依赖于对象，因此，可用普通指针实现。

```
// func static:
void (*funcPtr)() = &Point::showCount;    //&可选
funcPtr();    或者    (*funcPtr)();

// int static
int *ptr = &Point::count;                    //&不能少!
*ptr
```

- 综合案例：

```
#include <iostream>
using namespace std;

class example{
public:
    int id;
    static int num;
    example(int id) : id(id){
        num ++;
    }
}
```

```

    void checkid(){
        cout << id << endl;
    }

    static void checknum(){
        cout << num << endl;
    }
};

int example::num = 0;

int main(){
    example e1(10), e2(20);
    example *eptr = &e2;
    void (example::*funcptr1)() = &example::checkid;    // & necessary
    (e1.*funcptr1)();
    (eptr->*funcptr1)();    // both ok!    .* and ->* are new symbols!

    void (*funcptr2)() = &example::checknum;    // & unnecessary
    funcptr2();
    (*funcptr2)();
    (**funcptr2)();    //correct in grammar

    int example::*idptr = &example::id;    // & necessary
    cout << e1.*idptr << endl;
    cout << eptr->*idptr << endl;

    int *numptr = &example::num;    // & necessary
    cout << numptr << endl;    // addr
    cout << *numptr << endl;    // correct num

    return 0;
}

```

3 字符串string类

3.1 getline方式

- getline可以输入整行字符串（要包括string头文件），例如：
 - getline(cin, s2);
- 输入字符串时，可以使用其它分隔符作为字符串结束的标志（例如逗号、分号），将分隔符作为getline的第3个参数即可，例如：

– getline(cin, s2, ',');

3.2 sizeof, length, strlen一网打尽

```
#include <iostream>
#include <cstring>

using namespace std;

int main(){
    string str1 = "ldkzsznb";

    string str2;
    cin >> str2;        //ldkzsznb

    string str3;
    cin.ignore();        // clear "\n" in str2
    getline(cin, str3);   //ldkzsznb

    const char *str4 = "ldkzsznb"; // const is necessary in c++11

    const char str5[] = "ldkzsznb";

    cout << str1.length() << endl;    //8
    cout << sizeof(str1) << endl;      //32

    cout << str2.length() << endl;    //8
    cout << sizeof(str2) << endl;      //32

    cout << str3.length() << endl;    //8
    cout << sizeof(str3) << endl;      //32

    cout << sizeof(str4) << endl;      //8  指针本身占用的大小
    cout << sizeof(*str4) << endl;     //1
    cout << strlen(str4) << endl;      //8

    cout << sizeof(str5) << endl;      //9  整个数组所占用的内存大小
    cout << sizeof(*str5) << endl;     //1
    cout << strlen(str5) << endl;      //8

    return 0;
}
```

4 动态内存分配new

```
Point *ptr1 = new Point;    //调用缺省构造函数
ptr1->setXY(1, 2);
delete ptr1;                //删除对象，自动调用析构函数

Point *ptr = new Point[2]; //调用缺省构造函数
ptrArray[0].setXY(1, 2);
delete[] ptr;

//c++11之后，这种都可以！
Point* ptr1 = new Point[2]{{1, 2}, {3, 4}};
Point* ptr2 = new Point[2]{Point(7, 8), Point(9, 10)};
```

- new 类型名T：调用缺省构造函数，如果构造函数没有初始化类变量，类变量值不可预测；
- new 类型名T ()：调用缺省构造函数，如果构造函数没有初始化变量，用0/NULL初始化变量/对象。
- 将动态数组封装成类：

```
class ArrayOfPoints {        //动态数组类
public:
    ArrayOfPoints(int size) : size(size) {
        points = new Point[size];
    }
    ~ArrayOfPoints() {
        cout << "Deleting..." << endl;
        delete[] points;
    }
    Point& element(int index) {
        assert(index >= 0 && index < size);
        return points[index];
    }
private:    Point *points; //指向动态数组首地址
            int size;      //数组大小
};
```

- 断言assert：如果它的条件返回错误，则终止程序执行。assert仅仅在debug版中起作用。
- 和malloc的对比：
 - new 是操作符， malloc 是函数；
 - new 分配内存后会自动调用构造函数， malloc 仅分配原始内存；

- `malloc` 分配失败时返回 `NULL`, `new` 抛出 `bad_alloc` 异常;
- `new` 不需要强制类型转换, `malloc` 返回 `void*` 需要强制转换。

5 用vector创建数组对象

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <numeric> // for accumulate, iota
#include <iterator> // for back_inserter

using namespace std;

// 打印vector的辅助函数
template<typename T>
void printVector(const string& name, const vector<T>& vec) {
    cout << name << ": ";
    for (const auto& element : vec) {
        cout << element << " ";
    }
    cout << " | 大小: " << vec.size()
         << ", 容量: " << vec.capacity() << endl;
}

int main() {
    cout << "==== Vector初始化方式 =====" << endl;

    // 1. 默认初始化 - 空vector
    vector<int> v1;
    printVector("v1(默认初始化)", v1);

    // 2. 指定大小初始化
    vector<int> v2(5); // 5个元素, 默认值0
    printVector("v2(大小5)", v2);

    // 3. 指定大小和初始值
    vector<int> v3(5, 100); // 5个100
    printVector("v3(5个100)", v3);

    // 4. 列表初始化 (C++11)
    vector<int> v4 = {1, 2, 3, 4, 5};
    printVector("v4(列表初始化)", v4);

    // 5. 拷贝初始化
```

```

vector<int> v5(v4);
printVector("v5(拷贝v4)", v5);

// 6. 范围初始化
vector<int> v6(v4.begin() + 1, v4.end() - 1); // 2,3,4
printVector("v6(范围v4[1]-v4[3])", v6);

// 7. 移动初始化 (C++11)
vector<int> v7(move(vector<int>{10, 20, 30}));
printVector("v7(移动初始化)", v7);

// 访问元素
cout << "v4[0] = " << v4[0] << endl; // 运算符[]
cout << "v4.at(1) = " << v4.at(1) << endl; // 带边界检查
cout << "v4.front() = " << v4.front() << endl; // 第一个元素
cout << "v4.back() = " << v4.back() << endl; // 最后一个元素

// 修改元素
//v4[0] = 100;
//v4.at(1) = 200;

// 添加元素
//v4.push_back(1);

// 插入元素
//v4.insert(v4.begin() + 1, 99); // 在位置1插入99
//v4.insert(v4.end(), {7, 8, 9}); // 末尾插入多个

// 删除元素
//v4.dynamicVec.pop_back(); // 删除最后一个
//v4.printVector("pop_back后", dynamicVec);
//v4.erase(v4.begin() + 2); // 删除位置2的元素
//v4.erase(v4.begin() + 1, v4.begin() + 3); // 删除范围
//v4.clear(); check: v4.size(),capacityVec.empty()

// 排序
//sort(v4.begin(), v4.end()); //algorithm

// 反转
//reverse(v4.begin(), v4.end()); //algorithm

// 查找
//auto it = find(v4.begin(), v4.end(), 5); //algorithm

// 统计
//int countFives = count(v4.begin(), v4.end(), 5);

```



```
// 累加
//int sum = accumulate(v4.begin(), v4.end(), 0);

// 二维vector初始化
vector<vector<int>> matrix1 = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

// iterator, later in concrete
for (auto it = v4.begin(); it != v4.end(); ++it) {
    cout << *it << " ";
}
```

6 cast类型

转换类型	安全性	主要用途	编译期/运行期	典型场景
static_cast	中等	相关类型转换	编译期	int ->float
dynamic_cast	高	多态类型安全向下转型	运行期	Dog -> Animal
const_cast	低	修改const	编译期	const int* -> int*
reinterpret_cast	极低	底层无关类型转换	编译期	int* -> char*

第七章：类的继承

1 基类和派生类

💡 概念

1. 组合关系：has-a关系，就是汽车和发动机的关系，这直接指导了类的嵌套；
 2. 继承关系：is-a关系，就是学生和人的关系，是一个递进的描述。
- 基类（或父类）、派生类、直接基类、间接基类；**所有可以使用基类的对象的地方均可以使用派生类的对象。**
 - 单继承、多继承；
 - 继承的目的：实现代码重用。
 - 派生的目的：当新的问题出现，原有程序无法解决（或不能完全解决）时，需要对原有程序进行改造。
 - 继承的组成部分：
 - 规定访问控制权限；
 - 初始化，注意顺序！
 - 吸收基类成员：派生类实际上就包含了它的全部基类中除构造函数和析构函数之外的所有成员。
 - 改造基类成员：派生类声明了一个和某基类成员同名的新成员（如果是成员函数，则参数表也要相同，参数不同的情况属于重载），派生的新成员就覆盖了外层所有同名成员
 - 添加新的成员；

2 访问控制

- 公有继承public: 基类的public和protected成员的访问属性在派生类中保持不变，但基类的private成员不可直接访问。使用上，感觉将父类完全拿来使用！
- 私有继承private: 基类的public和protected成员都以private身份出现在派生类中，但基类的private成员不可直接访问。
- 保护继承protected: 基类的public和protected成员都以protected身份出现在派生类中，但基类的private成员不可直接访问。

⚠ 注意

protected的意义：

- 对建立其所在类对象的模块来说，它与 `private` 成员的性质相同。
- 对于其派生类来说，它与 `public` 成员的性质相同。

派生类没有继承基类的`private`成员：这句话是**错误的**，虽然不可以直接访问，但是数据都被继承来了，需要基类部分函数进行访问罢了。**基类的所有成员变量都可以被派生类继承**：这句话是**正确的**。

3 类型兼容规则

- 一个公有派生类的对象在使用上可以被当作基类的对象，反之则禁止。
- 派生类的对象可以隐含转换为基类对象。
- 派生类的指针可以隐含转换为基类的指针。
- **切片**的时候一律减少成基类原本的样子，包括虚函数，只有用指针引用的时候，虚函数发生作用。

```
#include <iostream>
#include <string>

class Animal {
public:
    std::string name;
    Animal(std::string n = "Unknown Animal") : name(n) {}

    // 非虚函数
    void makeSound() {
        std::cout << name << " makes a generic animal sound." <<
std::endl;
    }

    // 虚函数
    virtual void eat() {
        std::cout << name << " eats food." << std::endl;
    }

    // 另一个非虚函数，在派生类中重新定义
    void identify() {
        std::cout << "I am an " << name << " (Base)." << std::endl;
    }
};

class Dog : public Animal {
public:
```

```

        std::string breed;
        Dog(std::string n = "Dog", std::string b = "Mixed") : Animal(n),
        breed(b) {}

        // 重新定义非虚函数 (隐藏 Base::makeSound)
        void makeSound() {
            std::cout << name << " barks!" << std::endl;
        }

        // 重写虚函数 (多态行为)
        void eat() override {
            std::cout << name << " (a " << breed << " dog) eats kibble."
            << std::endl;
        }

        // 重新定义非虚函数 (隐藏 Base::identify)
        void identify() {
            std::cout << "I am a " << breed << " " << name << "
            (Derived)." << std::endl;
        }

        void fetch() {
            std::cout << name << " fetches the ball." << std::endl;
        }
    };

int main() {
    Dog myDog("Buddy", "Golden Retriever");

    std::cout << "--- 1. 直接通过派生类对象调用 ---" << std::endl;
    myDog.makeSound(); // Dog::makeSound()
    myDog.eat();        // Dog::eat()
    myDog.identify();    // Dog::identify()
    myDog.fetch();       // Dog::fetch()

    std::cout << "\n--- 2. 对象切片 (Base b_obj = myDog) ---" <<
    std::endl;
    Animal genericAnimal = myDog; // 切片发生!
    genericAnimal.makeSound();    // 调用 Animal::makeSound(), 而非
    Dog::makeSound()
    genericAnimal.eat();          // 调用 Animal::eat(), 而非
    Dog::eat() (因为 genericAnimal 是一个独立的 Base 对象了)
    genericAnimal.identify();     // 调用 Animal::identify()
    // genericAnimal.fetch();     // 编译错误: Animal 对象没有 fetch()

    std::cout << "\n--- 3. 通过基类指针/引用实现多态 (Base* ptr =
    &myDog) ---" << std::endl;
    Animal* animalPtr = &myDog; // 没有切片

```

```

    animalPtr->makeSound();    // 调用 Animal::makeSound(), 静态绑定
    animalPtr->eat();           // 调用 Dog::eat(), 动态绑定 (虚函数)
    animalPtr->identify();      // 调用 Animal::identify()
    // animalPtr->fetch();      // 编译错误: Animal 指针没有 fetch()

    return 0;
}

```

4 派生类的构造、析构函数

①先虚基类，然后调用基类构造函数，调用顺序按照它们被继承时声明的顺序（从左向右）。

②对本类成员初始化列表中的基本类型成员和对象成员进行初始化，初始化顺序按照它们在类中声明的顺序。

③析构函数的顺序和构造函数正好相反。

```

class Derived: public Base2, public Base1, public Base3 {
//派生新类Derived, 注意基类名的顺序
public:           //派生类的公有成员
    Derived(int a, int b, int c, int d): Base1(a), member2(d),
member1(c), Base2(b)
    { }
    //注意基类名的个数与顺序, //注意成员对象名的个数与顺序
private:        //派生类的私有成员对象
    Base1 member1;
    Base2 member2;
    Base3 member3;
};

int main() {
    Derived obj(1, 2, 3, 4);
    return 0;
}

/*
    constructing Base2 2
    constructing Base1 1
    constructing Base3 *
    constructing Base1 3
    constructing Base2 4
    constructing Base3 *

```

- 关于复制构造函数： `C::C(const C &c1): B(c1) {...}` `//c1为什么不是B类引用？` 类型兼容规则
- `delete`的构造函数，在派生类中同样被删除！
- 概念上：**构造函数和析构函数不能被继承**

5 派生类成员的标识与访问

- 可以用“`::`”表示作用域；
- 二义性问题如何解决：虚函数、同名隐藏、类名限定、虚基类
 - 虚函数：定义virtual函数，这样可以声明在后面override，最重要的是可以用过指针或者引用去切片了！
 - 同名隐藏：在C类中创建一个f()函数，内部调用A::f() 或 B::f()，这样就覆盖了。
 - 类名限定： `c1.A::f()` 或 `c1.B::f()`

```
#include <iostream>
using namespace std;
class A{
public:
    void func(){
        cout << "classA::func" << endl;
    }
    virtual void funcv(){
        cout << "classA::funcv" << endl;
    }
};

class B{
public:
    void func(){
        cout << "classB::func" << endl;
    }
    virtual void funcv() {
        cout << "classB::funcv" << endl;
    }
};

class C : public A, public B{
public:
```

```

    void func(){
        // 同名隐藏
        A::func();
    }
    void funcv() override {
        cout << "classC::funcv" << endl;
    }
};

int main(){
    C x;
    x.func();

    C y;
    y.A::func();
    y.B::func();    //类名限定

    C z;
    z.funcv();
    B z1 = z;
    z1.funcv();    //classB::funcv
    B* z2 = &z;
    z2->funcv();    //classC::funcv
    B &z3 = z;
    z3.funcv();    //classC::funcv

    return 0;
}

```

- 虚基类：主要解决菱形继承中二义性问题。否则产生内存冗余和访问歧义。

```

#include <iostream>
using namespace std;

class Base0 {
public:
    Base0(int var) : var0(var) { }
    int var0;
    void fun0() { cout << "Member of Base0" << endl; }
};

class Base1: virtual public Base0 {
public:
    Base1(int var) : Base0(var) { }
    int var1;
};

class Base2: virtual public Base0 {
public:

```

```

        Base2(int var) : Base0(var) { }
        int var2;
};
class Derived: public Base1, public Base2 {
public:
    Derived(int var) : Base0(var), Base1(var), Base2(var) { }
    int var;
    void fun() { cout << "Member of Derived" << endl; }
};

int main() {
    Derived d(1);
    d.var0 = 2;
    d.fun0();
    return 0;
}

```

- 注意构造函数的执行顺序:

```

class Base {
public:
    Base() { cout << "Base()" << endl; }
};

class A : virtual public Base {
public:
    A() { cout << "A()" << endl; }
};

class B : virtual public Base {
public:
    B() { cout << "B()" << endl; }
};

class C : public A {
public:
    C() { cout << "C()" << endl; }
};

class D : public B {
public:
    D() { cout << "D()" << endl; }
};

class Final : public C, public D { // 复杂菱形
public:
    Final() : Base() {
        cout << "Final()" << endl;
    }
};

```

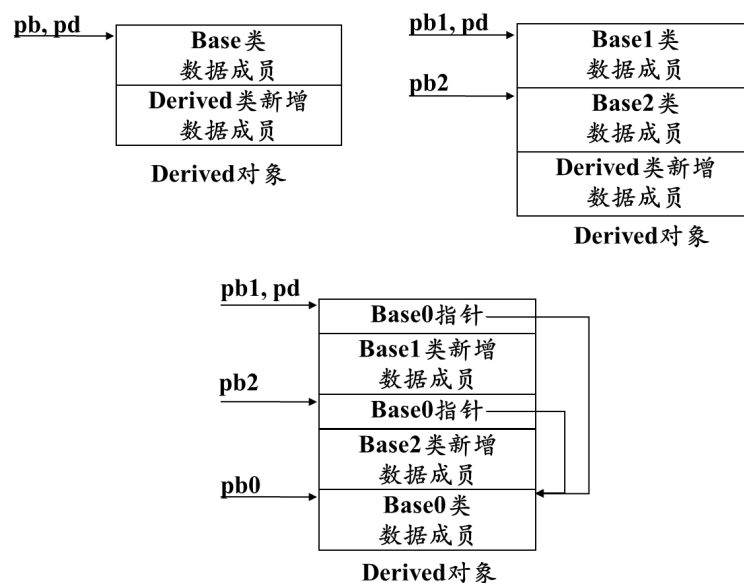


```

    }
};
/*
Base()      ← 虚基类由Final构造，永远是最先考虑的
A()         ← 虚基类A分支
C()         ← A的派生类
B()         ← 虚基类B分支
D()         ← B的派生类
Final()     ← 最终派生类
*/

```

- 内存布局：单继承、多继承、虚基类继承



① 注意

这就不得不提到虚函数的内存情况，虚函数的目的是多态，他需要维护一个虚函数表。

- 空类在内存中至少有1字节用于标识，如果涉及虚函数，则至少需要一个4字节的vptr。
- vtable 的结构和内容（即一个类有哪些虚函数，它们在 vtable 中的相对位置，以及它们默认指向哪个实现）是在编译时固定好的。
- 对象的 vptr 会在运行时被初始化，使其指向其实际类类型对应的 vtable。他会存储在数据成员的区域里面。
- 在实际调用虚函数时，运行时会使用对象的 vptr 来找到正确的 vtable，然后使用编译时确定的槽位信息来调用正确的函数。
- 因此：Base* ptr = new Derived(); 可以保留原始的对象函数；Base b_obj = derived_obj; 多态性会破坏。
- 派生类对象的内存布局中，包含了基类子对象的所有数据成员。

6 基类向派生的转换（向下转换 Downcasting）

- 原则上是不安全的，但是这里用static_cast转换，默认安全！指针和引用是可以的。

```
Base *pb = new Derived();
Derived *pd = static_cast<Derived *>(pd);
Derived d;
Base &rb = d;
Derived &rd = static_cast<Derived &>(rb);
```

- 如果A类型是B类型的虚拟基类，A类型指针无法通过static_cast隐含转换为B类型的指针。
- 根据内存的案例，理解为什么这段代码是错误的！

```
Derived *pd = new Derived();
void *pv = pd; //将Derived指针转换为void指针
Base2 *pb = static_cast<Base2 *>(pv);
```

- 更安全更灵活的基类向派生类转换方式——dynamic_cast（也是downcast），将在下一讲介绍
- 向上转换 (Upcasting)：将派生类对象的指针或引用转换为其基类对象的指针或引用。

第八讲：多态性

1 基本概念

- 编译时多态：也称为静态绑定、静态联编
 - 强制多态，这就是**强制类型转换**，之前学过；
 - 重载多态，包括函数重载和**运算符重载**，前者之前学过；
 - 参数多态，主要指的是**函数模板、类模板**；
 - **好处：调用速度更快、运行效率更高**
- 运行时多态：也称为动态绑定、动态联编
 - 包含多态，主要是**虚函数**的多态；
 - **好处：编程更灵活、代码扩展性好**

2 运算符重载

2.1 概念

- 基本数据类型重载了运算符函数`operator+()`。这些重载形式如：`operator+(int, int)`；
- 只能够重载C++中已经有的运算符；
- 重载之后运算符的优先级和结合性都不会改变；
- `operator`重载运算符：由关键字和要重载算符组成函数名；
- 可以重载：`+-*/%^&|~>><<<<=>!===!&&||+=-=*=/=%<<=>>=&|=^=++--[ ]()>new new[ ]delete delete[ ]`
 - 以下运算符只允许用成员函数重载：`=() [ ] new delete`
- 不能重载：`..*::?: sizeof`

2.2 规则

- `aa @ 或 @ aa`：
 - 成员函数：`aa.operator @()`
如果你有一个 `MyClass` 类的对象 `myObj`，并重载了前置自增运算符 `++` 为成员函数，那么当执行 `++myObj` 时，实际调用的是 `myObj.operator++()`。
 - 友元函数：`operator @(aa)`

- aa @ bb:
 - 成员函数: aa . operator @ (bb)
 - 友元函数: operator @ (aa,bb)

2.3 一个重载的类完整实现

- 构造函数，一定记得包括复制构造函数（建议重载函数的参数用引用能避免一次复制）；
- 析构函数；
- 重载函数：
 - 主要是成员函数，用于处理主要的重载；还引入了友元函数，用于处理1+A这种交换问题；
 - 返回类型有的是对象，有的是对象的引用，这个要看该操作是否需要对象本身。
 - 加法操作：a+b+c，a+b运算后和a和b自身已经没有关系了，就是一个新的变量，因此是对象；
 - 前置自增操作：(++a)+b，在++a中实现了a的递增，进而可以继续使用a的引用，当然可以不用；你可能还会考虑(a++)++，说a++之后再++，就必须是引用呀！但是事实上这句话编译会报错，因为a++是右值。
 - 后置自增操作：(a++)+b，在a++的时候新建了局部变量，只能值返回。如果引入new的方式，不方便实现delete。
 - 赋值操作：返回的必须是引用，因为(a=b)++，赋值表达式的结果是左值；
 - 要注意指针悬挂问题，这个和前面的复制构造函数的处理类似！
 - 不能被继承！
 - 左移操作：返回的必须是引用，因为cout这周ostream需要进一步运行；
 - 参数：建议不会变化的就直接const + &保护！
 - 函数体：不建议使用new！
- 继承问题：Derived result2 = d1 + d2; 如果只有Base有定义加法，则result2退化为Base类型，这里会报错！

```
#include <iostream>
using namespace std;
class Complex {
private:
    double a;
```

```

    double b;
public:
    // constructor and distructor
    Complex():a(0),b(0){}
    Complex(double a): a(a),b(0){}
    Complex(double a, double b):a(a),b(b){}
    Complex(const Complex &other){
        a = other.a;
        b = other.b;
        cout << "copy" << endl;
    }
    ~Complex(){}

    //add
    Complex operator+(const Complex &other);
    Complex operator+(const int x);           // Complex + int
    friend Complex operator+(const int x, Complex y); // int + Complex,
can be const Complex &y!

    //self_add
    Complex& operator++();    //placed first!
    Complex operator++(int);  //placed second!

    //assign
    Complex& operator=(const Complex &ori);
    Complex& operator=(const int x);

    //print
    friend ostream& operator<<(ostream& os, const Complex &c);

};

Complex Complex::operator+(const Complex &other){
    return Complex(a + other.a, b + other.b);
}
Complex Complex::operator+(const int x){
    return Complex(a + x, b);
}
Complex operator+(const int x, Complex y){
    return y + x;
}

Complex& Complex::operator++(){    //++c
    this->a ++;
    return *this;
}
Complex Complex::operator++(int){    //c++
    Complex c = *this;

```

```

    ++(*this);
    return c;
}

Complex& Complex::operator=(const Complex &ori){
    a = ori.a;
    b = ori.b;
    return *this;
}

Complex& Complex::operator=(const int x){
    a = x;
    b = 0;
    return *this;
}

ostream& operator<<(ostream& os, const Complex &c){
    os << c.a << "+" << c.b << "i";
    return os;
}

int main(){
    Complex c1, c2(5), c3(10,15);
    cout << c2+c3 << endl; // three RVOs!
    cout << c3+2 << endl;
    cout << 9+c3 << endl; //copy, can be optimized!

    cout << c3++ << endl; //copy
    cout << c3 << endl;
    cout << ++c2 << endl;

    Complex c4;
    c4 = c3 + c2;
    cout << c4 << endl;
    c4 = 10;
    cout << c4 << endl;

    return 0;
}

```

2.4 小括号和中括号的重载（用法较少）

```

#include <iostream>
using namespace std;

class Adder{
public:
    int operator()(const int a, const int b){

```

```

        return a + b;
        cout << "operator()" << endl;
    }
    int func(const int a, const int b){
        return a + b;
        cout << "func1" << endl;
    }
};

class MyVector{
private:
    int* v;
public:
    MyVector(int size){
        v = new int(size);
    }
    ~MyVector(){
        delete[] v;
    }
    int& operator[](int num){
        return v[num];
    }
};

int main(){
    Adder add;
    cout << add(1, 2) << endl;
    cout << add.func(1, 3) << endl;

    MyVector v(10);
    v[2] = 10;
    cout << v[2] << endl;

    return 0;
}

```

3 虚函数

3.1 基本概念

- **虚函数**的定义是：可以被派生类复写的函数；**纯虚函数**的定义是：直接不写了，因为和派生类没什么共同点。

```
virtual void func() = 0; // 纯虚函数
virtual void func() {} // 普通虚函数
```

- **抽象类的定义是：至少含有一个纯虚函数的类，因此不能定义抽象类对象，但是如果从派生类cast为基类指针是允许的！**
- 构造函数不可以是虚函数；
- 如果一个类可能被用作基类，并且包含虚函数，那么它的析构函数**必须**声明为虚函数。如果它可能被用作基类，即使没有其他虚函数，也**强烈建议**声明为虚函数。

```
Base_NonVirtual* ptr_non_virtual = new Derived_NonVirtual(); // 通过
基类指针指向派生类对象
delete ptr_non_virtual;
```

- 假如不采用 virtual，则当 `delete ptr_non_virtual;` 被调用时，`ptr_non_virtual` 的静态类型是 `Base_NonVirtual*`。由于 `Base_NonVirtual` 的析构函数不是 virtual 的，编译器会执行**静态绑定**（编译时决定调用哪个函数）。它只会看到 `Base_NonVirtual` 的析构函数，并调用它。结果是：`Derived_NonVirtual` 类的析构函数**没有被调用**！
- 采用的话，当 `delete ptr_virtual;` 被调用时，`ptr_virtual` 的静态类型是 `Base_Virtual*`。但由于 `Base_Virtual` 的析构函数被标记为 virtual，编译器会执行**动态绑定**（运行时通过虚函数表 `vtable` 决定实际调用哪个函数）。运行时系统会识别出 `ptr_virtual` 实际指向的是一个 `Derived_Virtual` 类型的对象。因此，它会先调用**派生类 `Derived_Virtual` 的析构函数**，释放 `data` 内存。接着，**基类 `Base_Virtual` 的析构函数**会被自动调用。
- 如果不加 virtual，系统不知道这是一个派生类，他以为就是正常的一个析构，进而调用自己的析构函数(更底层的)，系统会进一步按照与构造函数相反的顺序执行析构函数。析构函数释放的都是自己开辟的额外内存，不存在重复释放的可能。
- 多态类型：有虚函数的类；
 - 多态类型支持运行时类型识别：需要理解为，编译的时候只知道函数是虚函数，只有运行的时候通过虚表才能确定是使用哪个函数（**这应该称之为动态绑定**）。
 - 多态类型对象占用额外的空间：虚表是空间，对象本身也需要额外的指针。
 - 然而有的时候动态绑定不够，因为需要在Base指针的基础上获得更多派生类的信息，这就需要**运行时类型识别**：

- `dynamic_cast`: `Derived1 *d = dynamic_cast<Derived1 *>(b);` 对于指针类型, `dynamic_cast` 失败时会返回 `nullptr` (题目中说只会抛出异常, 这部分存疑); 对于引用类型, 它会抛出 `std::bad_cast` 异常。只能用于**多态类型** (即至少包含一个虚函数的类)。
- `typeid`: 可以获得基类拓展后的真实信息。

```
#include <iostream>
#include <typeinfo>
using namespace std;

class Base {
public:
    virtual ~Base() { }
};

class Derived: public Base { };

void fun(Base *b) {
    //得到表示b和*b类型信息的对象
    const type_info &info1 = typeid(b);
    const type_info &info2 = typeid(*b);

    cout << "typeid(b): " << info1.name() << endl;
    cout << "typeid(*b): " << info2.name() << endl;
    //判断*b是否为Base类型
    if (info2 == typeid(Base))
        cout << "A base class!" << endl;
}

int main() {
    Base b;
    fun(&b);
    Derived d;
    fun(&d);
    return 0;
}
```

- 虚函数存在的价值可以理解为一个基类应该有的功能, 但是每个派生类的实现方式不一样, 因此做个说明。
- 关键字:
 - `virtual`说明的函数是虚函数, 被继承的虚函数可以不写`virtual`关键词;
 - `override`是显式告诉编译器的一种方式, 主要是性能的提升和编译时确保重写的虚函数名字不会敲错。
 - `final`是告诉编译器这个虚函数不打算继续拓展了, 不允许被重写! `final`也可以修饰一个类, 这就意味着这个类不能再继续派生了。

- 非静态成员函数、析构函数可以是虚函数；普通函数（非成员函数）、构造函数、内联成员函数、静态成员函数不能是虚函数；
 - 基类的析构函数最好标记为virtual，因为可能需要delete全部的地址空间，这里析构函数不需要重名。

3.2 代码实例

```
#include <iostream>
using namespace std;

class Shape{
private:
    string name;
public:
    Shape(string str) : name(str) {}
    virtual ~Shape(){
        cout << "destructing shape!" << endl;
    }
    virtual double getArea() = 0;
    virtual double getPerim() = 0;
};

class Rectangle : public Shape{
private:
    double x;
    double y;
public:
    Rectangle(string str, double x, double y) : x(x), y(y), Shape(str) {
    }
    virtual ~Rectangle() override {
        cout << "destructing rectangle!" << endl;
    }
    /*virtual*/ double getArea() override final {
        return x*y;
    }
    /*
        实际上，一旦基类中的函数被声明为 virtual，那么在派生类中重写 (override) 它
        时，virtual 关键字是可选的。它仍然是虚函数。一些程序员可能会加上它以清晰表明其
        虚函数性质，但现代 C++ 中，override 关键字已经足够明确地表达了这一点。
    */
    virtual double getPerim() override final {
        return 2*(x+y);
    }
};
```

```

int main(){
    Rectangle r("r1", 2.5, 4);
    cout << r.getArea() << endl;
    cout << r.getPerim() << endl;

    Shape *s = &r;
    cout << s->getArea() << endl;
    cout << s->getPerim() << endl;

    /* Shape x("error"); */
    return 0;
}

```

3.3 内存管理

- 每个对象有一个指向当前类的虚表的指针（虚指针vptr）；
- 虚表中有当前类的各个虚函数的入口地址；
- 举一个复杂的例子，如果有虚基类、虚函数，多重继承，一个对象长什么样：
 - (MostDerived) vptr
 - (Derived1) vptr
 - (Derived1) VBptr
 - int d1_data
 - (Derived2) vptr
 - (Derived2) VBptr
 - int d2_data
 - int md_data
 - (共享的 Base 子对象) vptr
 - int base_data

第九章：模板和群体数据

1 函数模板和类模板

1.1 定义方法

- `template <typename T1, typename T2, ...>`
 - `typename` 可以替换为 `class`，二者可以认为是等价的，编程中推荐使用 `typename`；
- `class Adder` 或者 `void func(int T){...}`
 - 类模板外实现成员函数: `void Adder<T>::func(int T){...}`
- 使用函数中，只需要正常调用即可，类中定义 `Adder<int> a1;`
- 类模板不能表示具体的数据类型，但类模板的实例化类是数据类型；**函数模板本身不是函数**。
 - 隐含实例化，函数模版可以不声明类型，自动推断；类模版不行，必须显式说明；
 - 显式实例化，这是因为在多文件系统中，模板不允许声明和定义分离，可以选择把定义写在声明文件中，也可以选择显式实例化在cpp文件末尾。
 - `template void outputArray(const int *array, int count);`
 - `template class Store<double>;`
- 特化模板的含义：在bool+32的情况下时，采用特殊的定义。还有偏特化，部分参数固定，它只适用于**类模板**，**不适用于函数模板**。

```
// 通用模板
template<typename T, int Size = 100>
class Stack {
private:
    T data[Size];
    int top;
public:
    Stack() : top(-1) {}
    void push(T item) { data[++top] = item; }
    T pop() { return data[top--]; }
    bool isEmpty() const { return top == -1; }
    bool isFull() const { return top == Size - 1; }
};

// 特化版本: Stack<bool, 32>
template <>
class Stack<bool, 32> {
private:
```

```

    unsigned int list;    // 用1个int存储32个bool
    int top;
public:
    Stack() : list(0), top(-1) {}

    // 特化的push: 用位操作
    void push(bool item) {
        assert(!isFull());
        ++top;
        list = (list << 1) | (item ? 1 : 0); // 左移后设置最低位
    }

    // 特化的pop: 用位操作
    bool pop() {
        assert(!isEmpty());
        bool result = (list & 1) == 1; // 获取最低位
        list >>= 1; // 右移一位
        --top;
        return result;
    }

    bool isEmpty() const { return top == -1; }
    bool isFull() const { return top == 31; } // 32位只能存32个bool
};

// 偏特化的一个案例! 不支持模板函数!
template<typename T1>
class Pair<T1, double> { // 只固定T2=double, T1保持模板
public:
    void identify() { cout << "偏特化Pair<T1, double>" << endl; }
};

```

- 重载：选择“最特殊的一个”。具体类型比模板类型更特殊，特化比模板更特殊！
- 在类的声明处 `<typename T=double>`，则可以写为 `Point<> point()`。
- 编译的时候可以使用模板元编程：

```
// 模板元编程：编译时计算
template<int N>
struct Factorial {
    static const int value = N * Factorial<N - 1>::value;
};

// 特化：递归终止条件
template<>
struct Factorial<0> {
    static const int value = 1;
};
```

- 模板模板参数（Template Template Parameter）中，必须使用 `class` 关键字

```
template <template <typename E> class Container> // 这里必须使用 class
class MyWrapper {
    // ...
};
```

- 函数模板实例化分为：**隐式实例化**（由编译器自动在调用处生成）和**显式实例化**（用 `template func<int>(int);` 手动要求编译器生成）。
- 实例化过程发生在**编译期**，最终生成的是普通函数的目标代码，链接时与其他函数一样处理。

1.2 函数模板代码案例

```
#include <iostream>
using namespace std;
template <typename T>
T abs(const T &num){
    if(num >= 0) return num;
    else return -num;
}

int main(){
    int a = 5;
    double b = -10;
    cout << abs(a) << endl;
    cout << abs(b) << endl;
    return 0;
}
```

1.3 类模板代码案例

```
#include <iostream>
#include <cstring>
using namespace std;
template <typename T>
class Adder{
public:
    Adder();
    ~Adder();
    T add(const T &x, const T &y);
};
template <typename T>
Adder<T>::Adder(){}

template <typename T>
Adder<T>::~~Adder(){}

template <typename T>
T Adder<T>::add(const T &x, const T &y){
    return x+y;
}

int main(){
    Adder<int> a1;    //与模板函数不同，模板类在使用时必须显式地指定类型参数
    Adder<string> a2;
    cout << a1.add(10, -50) << endl;
    cout << a2.add("happy", " birthday") << endl;
    return 0;
}
```

2 群体数据

- 分为线性群体和非线性群体，前者包括直接访问群体（数组）、顺序访问群体（链表）和索引访问群体。

2.1 数组类

```
#include <iostream>
using namespace std;

template <typename T>
class Array{
private:
```

```

    T* list;
    int size;
public:
    Array(int size);
    Array(const Array<T> &a);
    ~Array();
    Array<T>& operator=(const Array<T> &other);
    T& operator[](int i);
    const T& operator[](int i) const; //const Array
    operator T*();
    operator const T*() const;
    int getsize() const;
    void resize(int sz);
};

template <typename T>
Array<T>::Array(int size){
    this->size = size;
    this->list = new T(size);
}

template <typename T>
Array<T>::Array(const Array<T> &a){
    this->size = a.size;
    this->list = new T(size);
    for(int i = 0; i < size; i++){
        list[i] = a.list[i];
    }
}

template <typename T>
Array<T>::~~Array(){
    if(list != NULL) delete[] list;
    list = NULL;
}

template <typename T>
Array<T>& Array<T>::operator=(const Array<T> &other){
    if(&other != this){ //important!
        if(size != other.size){
            delete[] list;
            list = new T(other.size);
            size = other.size;
            for(int i = 0; i < size; i++) {
                list[i] = other.list[i];
            }
        }
    }
}

```



```

        return *this;
    }

    template <typename T>
    T& Array<T>::operator[](int i){
        return list[i];
    }

    template <typename T>
    const T& Array<T>::operator[](int i) const {
        return list[i];
    }

    template <typename T>
    Array<T>::operator T*(){
        cout << "hahah" << endl;
        return list;
    }

    template <typename T>
    Array<T>::operator const T*() const{
        return list;
    }

    template <typename T>
    int Array<T>::getsize() const{
        return size;
    }

    template <typename T>
    void Array<T>::resize(int sz){
        if(sz == size) return;
        T* newList = new T(sz);
        int n = (sz < size) ? sz : size;
        for(int i = 0; i < n; i++){
            newList[i] = list[i];
        }
        delete[] list;
        list = newList;
        size = sz;
    }

    // 模板参数推导规则导致不能写成template!
    void read(double *arr, int f, int l){
        for(int i = f; i <= l; i++){
            cin >> arr[i];
        }
    }

```

```

}

int main(){
    Array<double> arr(5);
    read(arr, 0, 4);
    for(int i = 0; i <= 4; i++){
        cout << arr[i] << endl;
    }
    // ...
    return 0;
}

```

- 思路很简单，但是上述代码的理解有难度！

2.2 链表类

- 单链表示例：

```

#include <iostream>
#include <cstring>
using namespace std;

template <typename T>
struct node {
    T text;
    node<T>* next;
};

//list
template <typename T>
class list{
private:
    node<T>* front;
    int size;
public:
    list():front(NULL), size(0) {}
    ~list(){
        node<T>* tmp;
        while(front != NULL){
            tmp = front->next;
            delete front;
            front = tmp;
        }
    }
    void insertFront(T newone){
        node<T> *tmp = new node<T>;
    }
}

```

```

        tmp->text = newone;
        tmp->next = front;
        front = tmp;
        size ++;
    }
    void deletePlace(int place){
        if (place == 0) {
            node<T>* tmp = front;
            front = front->next;
            delete tmp;
            return;
        }

        node<T>* prev = front;
        for (int i = 0; i < place - 1 && prev != nullptr; i++) {
            prev = prev->next;
        }

        if (prev == nullptr || prev->next == nullptr) {
            cout << "位置超出范围" << endl;
            return;
        }

        node<T>* tmp = prev->next;    // 要删除的节点
        prev->next = tmp->next;        // 跳过要删除的节点
        delete tmp;                  // 释放内存
    }

    void printList(){
        node<T> *tmp = front;
        while(tmp != NULL){
            cout << tmp->text << " ";
            tmp = tmp->next;
        }
        cout << endl;
    }
};

int main(){
    list<string> l1;
    l1.insertFront("happy");
    l1.insertFront("birthday!");
    l1.printList();
    l1.deletePlace(1);
    l1.printList();
    return 0;
}

```

2.3 栈类

```
#include <iostream>
using namespace std;
struct node{
    int text;
    node* next;
};

class my_stack{
private:
    int size;
    node* first;
public:
    my_stack():size(0), first(NULL){}
    ~my_stack(){
        node* tmp = first;
        if(first == NULL) {return;}
        else if(first->next == NULL) {
            delete first;
            return;
        }
        first = first->next;
        while(first != NULL && first->next != NULL){
            delete tmp;
            tmp = first;
            first = first->next;
        }
        delete first;
    }
    void push(int num){
        node* newone = new node;
        newone->next = first;
        newone->text = num;
        first = newone;
        size ++;
    }
    int pop(){
        node* tmp = first;
        int txt = tmp->text;
        first = first->next;
        size --;
        delete tmp;
        return txt;
    }
    void print_info(){
        if(first == NULL) return;
    }
};
```

```

        while(first->next != NULL){
            cout << first->text << " ";
            first = first ->next;
        }
        cout << first->text << endl;
    }
};

int main(){
    my_stack stack;
    stack.push(10);
    stack.push(5);
    cout << stack.pop() << endl;
    stack.push(8);
    stack.print_info();
    return 0;
}

```

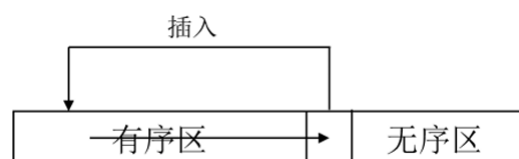
2.4 队列类

- 循环队列等。

2.5 组织：排序问题(内部)

2.5.1 插入排序

- 每一步将一个待排序元素按其关键字值的大小插入到已排序序列的适当位置上，直到待排序元素插入完为止。



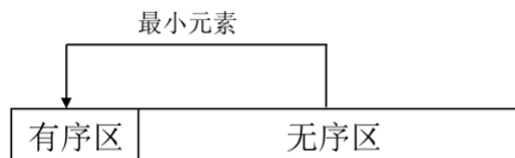
```

void InsertionSort(int* arr, int size){
    int tmp;
    int i,j;
    for(i = 1; i < size; i++){
        int j = i;
        int tmp = arr[i];
        while(j > 0 && tmp < arr[j - 1]){
            arr[j] = arr[j - 1];
            j --;
        }
        arr[j] = tmp;
    }
}

```

2.5.2 选择排序

- 选出最小的元素放到前面的有序区：



```

void SelectionSort(int* arr, int size){
    for(int i = 0; i < size; i++){
        int min = arr[i], pos = i;
        for (int j = i; j < size; j++){
            if(arr[j] < min) {
                min = arr[j];
                pos = j;
            }
        }
        int tmp = arr[i];
        arr[i] = arr[pos];
        arr[pos] = tmp;
    }
}

```

2.5.3 交换排序

- 最有名的是冒泡排序。

```
void BubbleSort(int* arr, int size){
    for(int j = size - 1; j >= 0; j --){
        for(int i = 0; i < j; i ++){
            if(arr[i] > arr[i + 1]){
                int tmp = arr[i + 1];
                arr[i + 1] = arr[i];
                arr[i] = tmp;
            }
        }
    }
}
```

2.6 组织：查找问题

2.6.1 顺序查找

```
int LinearCheck(int* arr, int size, int obj){
    for(int i = 0; i < size; i++){
        if(arr[i] == obj) return i;
    }
    return -1;
}
```

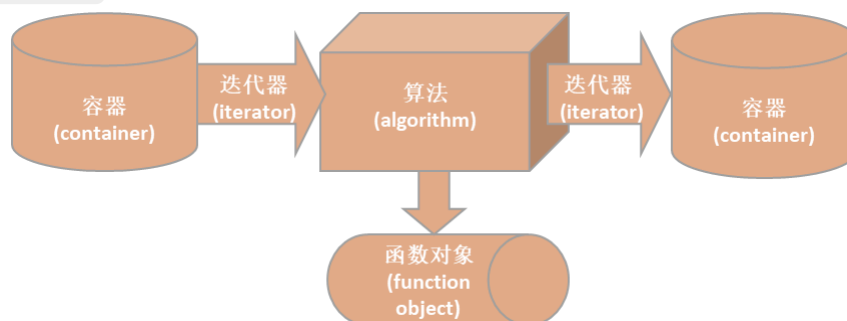
2.6.2 折半查找

```
int BinaryCheck(int* arr, int size, int obj){
    int low = 0, high = size - 1;
    while(low <= high){
        int mid = (low + high) / 2;
        if(obj == arr[mid]) return mid;
        else if(obj < arr[mid]) high = mid - 1;
        else low = mid + 1;
    }
    return -1;
}
```

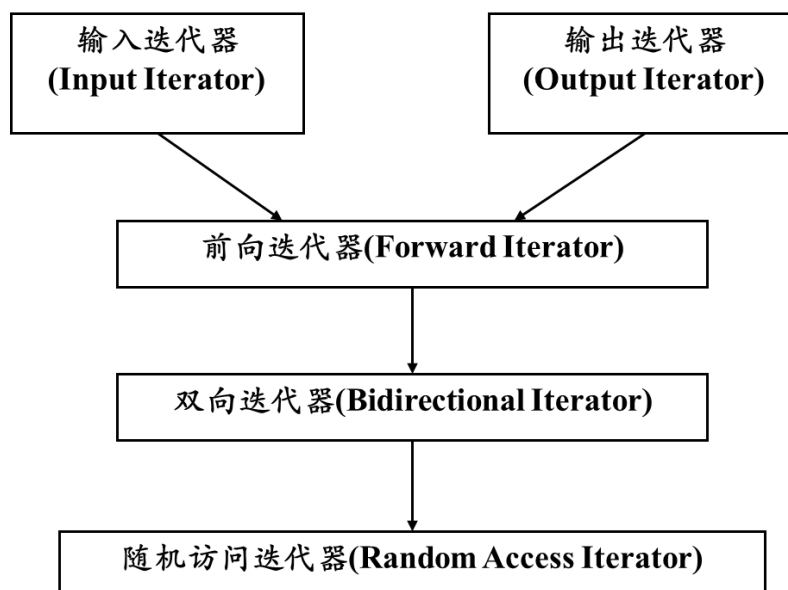
第十章： 泛型程序设计与C++语言标准模板库

1 泛型程序设计及STL的结构

- 标准模板库（Standard Template Library，简称STL）提供了一些非常常用的数据结构和算法；
- 组件：
 - 容器（container）：容纳、包含一组元素的对象。 `#include<vector>`
 - 迭代器（iterator）：迭代器是泛化的指针，提供了顺序访问容器中每个元素的方法。 `#include<iterator>`
 - 函数对象（function object）：任何普通的函数和任何重载了“()”运算符的类的对象都可以作为函数对象使用。 `#include<functional>`
 - 算法（algorithms）：举例为 transform 算法，举例见 PPT。 `#include<algorithm>`。



2 迭代器



- 输入迭代器：


```

istream_iterator<int> input_it(cin); // 从cin读取int
istream_iterator<int> eof;          // 结束标志
int value = *input_it; // 可以读
input_it++;             // 可以向前移动
// *input_it = 5;      // 错误! 不能修改

```

- 输出迭代器:

```

vector<int> v = {1, 2, 3, 4, 5};
// 输出流迭代器是典型的输出迭代器
ostream_iterator<int> output_it(cout, " ");
*output_it = 10; // 可以写
output_it++;     // 可以前进
// int x = *output_it; // 错误! 不能读
// 输出vector
copy(v.begin(), v.end(), output_it);

copy(s2.begin(), s2.end(), ostream_iterator<int>(cout, " "));
// 等价于:
auto dest = ostream_iterator<int>(cout, " ");
for(auto it = s2.begin(); it != s2.end(); ++it) {
    *dest = *it; // 关键: 解引用并赋值
    ++dest;     // 移动到下一个位置
}

transform(istream_iterator<double>(cin), istream_iterator<double>
(), ostream_iterator<double>(cout, "\t"),
square);

```

- 前向迭代器就是前两者的和, 双向是在其基础上允许双向。
- 一个案例:

```

// 头部分省略
int main() {
    istream_iterator<int> i1(cin), i2; //建立一对输入流迭代器
    vector<int> s1(i1, i2);           //通过输入流迭代器从标准输入流中输入数据
    sort(s1.begin(), s1.end());       //将输入的整数排序
    deque<int> s2;
    //以下循环遍历s1
    for (vector<int>::iterator iter = s1.begin(); iter != s1.end();
++iter)
    {
        if (*iter % 2 == 0)           //偶数放到s2尾部

```

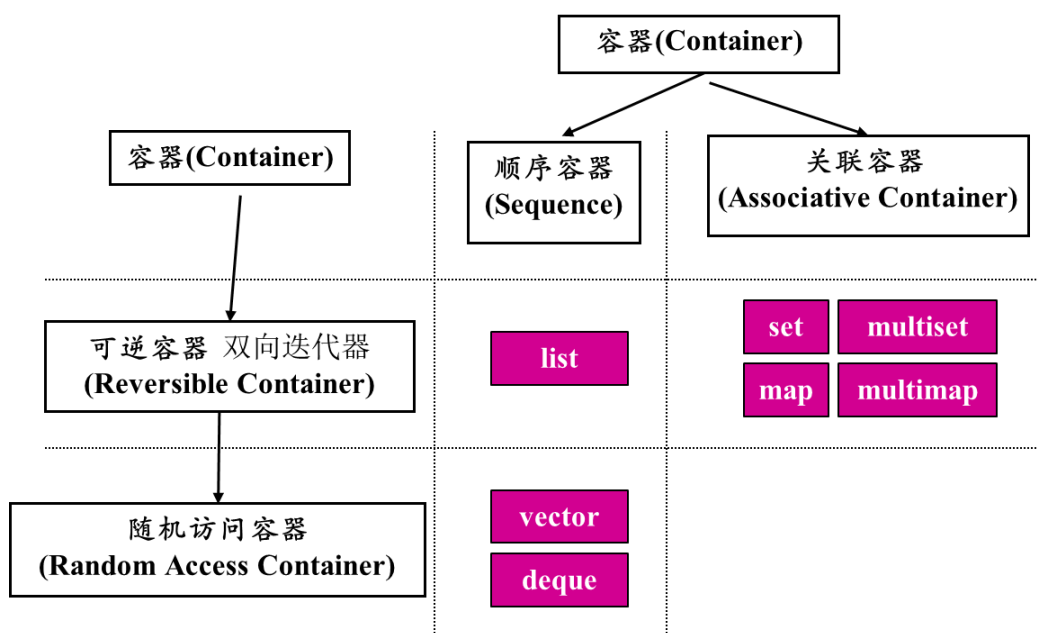
```

        s2.push_back(*iter);
else
    //奇数放到s2首部
    s2.push_front(*iter);
}
//将s2的结果输出
copy(s2.begin(), s2.end(), ostream_iterator<int>(cout, " "));
cout << endl;
return 0;
}

```

- 两个迭代器表示一个区间：[p1, p2)

3 容器



- 容器的第一种分类方式：
 - **可逆容器**：只能自增自减的迭代器
 - 双向迭代器 (Bidirectional Iterator)：支持 `++` 和 `--` 操作。例如 `std::list`、`std::set`、`std::map` 提供的迭代器就是双向迭代器。你可以向前或向后移动一步。可以用 `advance` 和 `distance` 函数处理更复杂的功能。
 - 前向迭代器 (Forward Iterator)：比双向迭代器更弱，只能自增 `++`，不能自减 `--`。例如 `std::forward_list` 提供的迭代器。
 - 输入/输出迭代器 (Input/Output Iterator)：这是最弱的迭代器，主要用于一次性遍历（读或写），通常在算法中使用，而不是容器直接提供给用户。

- **随机访问容器**：可以随机访问的迭代器。可以进行 `+n`、`-n`、`<`、`>`、`[]` 等操作的迭代器。它除了支持双向迭代器的所有操作外，还可以像指针一样进行算术运算，直接跳转到任意位置。例如 `std::vector`、`std::deque`、`std::string` 和普通数组的迭代器就是随机访问迭代器。
- **iterator 和 const_iterator**：几乎所有标准容器都提供这两种迭代器，分别用于可变和不可变地访问元素。
- **reverse_iterator**：只有当容器提供了**双向迭代器 (Bidirectional Iterator)** 或更强（如随机访问迭代器）时，才能提供 `reverse_iterator`。这意味着 `std::list`、`std::vector`、`std::map` 等都有 `reverse_iterator`，而 `std::forward_list` 没有 `reverse_iterator`，因为它只提供前向迭代器。
- 另一种分类方式：
 - **顺序容器 (Sequence Containers)**：
 - 根据元素在容器中的**插入顺序**来存储和访问元素，或者说元素的位置决定了它们的顺序。
 - 包括 `std::vector`、`std::deque`、`std::list`、`std::forward_list`、`std::array`、`std::string`。
 - **关联容器 (Associative Containers)**：
 - 根据元素的**键 (key)** 来存储和访问元素，键值是排序的。
 - 包括 `std::set`、`std::multiset`、`std::map`、`std::multimap`。
 - **无序关联容器 (Unordered Associative Containers)**：
 - `std::unordered_set`、`std::unordered_multiset`、`std::unordered_map`、`std::unordered_multimap`。它们基于哈希表，提供平均 $O(1)$ 的查找速度，但元素没有特定的顺序。

3.1 顺序容器

3.1.1 接口

- 构造函数 `vector<int> arr = {1,4,5,7};`
- 赋值函数：assign
- 插入函数：
 - `insert(iterator pos, const T& v)`，在pos位置插入后，返回新插入元素的迭代器；
 - `push_front`（只对list和deque），`push_back`，
 - `emplace_front`、`emplace` 和 `emplace_back`
- 删除函数：erase, clear, `pop_front`（只对list和deque），`pop_back`
- 首尾元素的直接访问：front, back
- 改变大小：resize

3.1.2 vector向量容器

- 和sort结合:

```
bool cmp(const int& a, const int& b){
    return a > b ? true : false;
}

int main (){
    vector<int> vec = {1, 4, 2, 7};
    sort(vec.begin(), vec.end(), cmp);
    for(auto x : vec) cout << x << " ";
    cout << endl;
}
```

- 常见操作:

```
vector<int> v = {5, 2, 8, 1, 9, 3, 2, 5};

// 1. 排序
sort(v.begin(), v.end()); // 升序
// v = {1, 2, 2, 3, 5, 5, 8, 9}

sort(v.begin(), v.end(), greater<int>()); // 降序

// 2. 反转
reverse(v.begin(), v.end());

// 3. 查找
auto it = find(v.begin(), v.end(), 8);
if(it != v.end()) {
    cout << "找到了8, 位置: " << (it - v.begin()) << endl;
}

// 4. 统计
int count5 = count(v.begin(), v.end(), 5);
cout << "5出现了" << count5 << "次" << endl;

// 5. 求和
int sum = accumulate(v.begin(), v.end(), 0);
cout << "总和: " << sum << endl;

// 6. 最大最小值
int max_val = *max_element(v.begin(), v.end());
int min_val = *min_element(v.begin(), v.end());
cout << "最大值: " << max_val << ", 最小值: " << min_val << endl;
```

```
// 7. 去重 (需要先排序)
sort(v.begin(), v.end());
auto last = unique(v.begin(), v.end());
v.erase(last, v.end()); // 删除重复元
```

- 遍历时删除操作:

```
vector<int> v = {1, 2, 3, 4, 5};

// 错误示例: 在遍历时删除元素
for(auto it = v.begin(); it != v.end(); it++) {
    if(*it == 2) {
        v.erase(it); // 删除后, it失效!
        // 下次使用it会导致未定义行为
    }
}

// 正确做法1: 使用返回值
vector<int> v2 = {1, 2, 3, 4, 5};
for(auto it = v2.begin(); it != v2.end(); ) {
    if(*it == 2) {
        it = v2.erase(it); // erase返回下一个有效迭代器
    } else {
        it++;
    }
}

// 正确做法2: 从后往前
vector<int> v3 = {1, 2, 3, 4, 5};
for(auto it = v3.end() - 1; it >= v3.begin(); it--) {
    if(*it == 2) {
        v3.erase(it);
    }
}
```

3.1.3 deque双端队列

- 功能更多, 可以在队列前面高效的插入、弹出。
- 编程的时候可以用它。

3.1.4 list列表容器

- 不支持随机访问。
- 支持splice结合操作 (特有): `s1.splice(p, s2, q1, q2)`: 将s2中[q1, q2)移动到s1中p所指向元素之前

3.1.5 string字符串容器

见“STL常见用法总结”。

3.1.6 适配器的概念

- "适配器" 是一种**设计模式**。在 C++ 标准库中，它特指那些**封装了现有类或接口，并提供一个不同接口**的类。你可以把适配器想象成一个插头转换器。它不会改变插头（底层对象）本身，只是改变了它露出来的**接口**，让它能适配不同的插座（期望的接口）。
- 常见的**插入迭代器**：包括前插迭代器（front_inserter）、后插迭代器（back_inserter）和任意位置插入迭代器（inserter）
 - `std::back_inserter`：适配 `push_back()`。
 - `std::front_inserter`：适配 `push_front()`。
 - `std::inserter`：适配 `insert(pos, value)`。

```
std::vector<int> source = {1, 2, 3};
std::vector<int> target_dynamic; // 原本无法直接拷贝到空 vector

// 使用 back_inserter 将 vector 的 push_back 适配成输出迭代器
std::copy(source.begin(), source.end(),
std::back_inserter(target_dynamic));
// target_dynamic 现在是 {1, 2, 3}，并且它的 push_back 被调用了三次
```

- 栈 (Stack) 和 队列 (Queue) 是适配器;

3.1.7 栈stack适配器

- `stack example; example.empty(); example.size(); example.top(); example.push(int); example.pop();`

3.1.8 队列queue适配器

- `queue example; example.empty(); example.size(); example.front(); example.back(); example.push(int); example.pop();`

3.1.9 优先级队列priority_queue适配器

- 最大堆： `priority_queue<int> pq;`
- 最小堆： `priority_queue<int, std::vector<int>, std::greater<int>> min_pq;`
- 自定义数据结构：这是最大堆在类里面的实现。

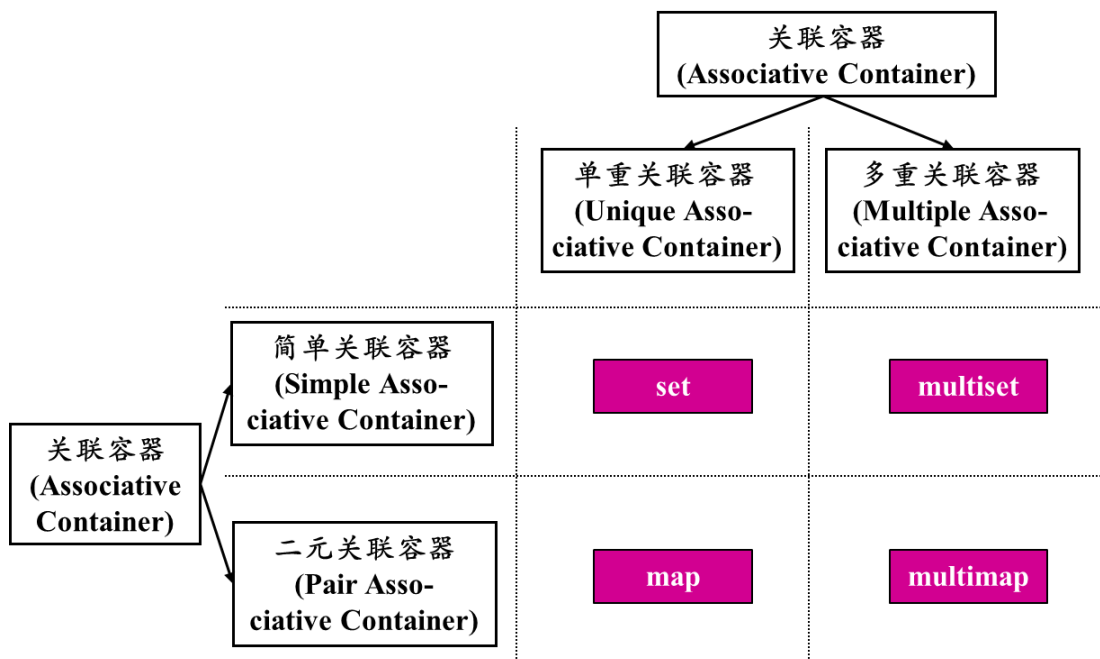
```
bool operator<(const Person& other) const {
    return age < other.age; // 年龄小的“小于”年龄大的（在最大堆中，年龄
    越大越靠前）
}
```

- 如果需要在类外面实现，也可以。

```
struct ComparePlayersByScoreAsc {
    bool operator()(const Player& p1, const Player& p2) const {
        return p1.score > p2.score; // 如果 P1 分数更高，则 P1 优先
        级低于 P2
    }
};

priority_queue<Player, std::vector<Player>,
ComparePlayersByScoreAsc> min_score_pq;
```

3.2 关联容器



- 单重关联容器(set和map): 键值是唯一的，一个键值只能对应一个元素
- 多重关联容器(multiset和multimap): 键值是不唯一的，一个键值可以对应多个元素
- 简单关联容器(set和multiset)
 - 容器只有一个类型参数，如set、multiset，表示键类型
 - 容器的元素就是键本身
- 二元关联容器(map和multimap)

- 容器有两个类型参数，如map<K,V>、multimap<K,V>，分别表示键和附加数据的类型
- 容器的元素类型是pair<K,V>，即由键类型和元素类型复合而成的二元组

3.2.1 接口

- 插入：insert
- 删除：erase
- 查找：find
- 定界：lower_bound第一个不小于、upper_bound第一个大于、equal_range
 - 作为类函数是特有的，其他的vector等也可以如下使用，引入algorithm头文件。

```
set<int> s = {10, 20, 20, 20, 30, 40, 50}; // set会自动去重

// 1. 成员函数版本 (更高效)
auto it = s.upper_bound(20);
cout << "第一个 > 20: " << *it << endl; // 30

// 2. 通用算法版本 (也可以用, 但效率一样)
auto it2 = upper_bound(s.begin(), s.end(), 20);
cout << "通用算法: " << *it2 << endl; // 30

// 3. 范围查询
int left = 15, right = 35;
auto start = s.lower_bound(left);
auto end = s.upper_bound(right);

cout << "在[" << left << "," << right << "]"之间: ";
for(auto it = start; it != end; ++it) {
    cout << *it << " "; // 20 30
}
```

```
vector<int> v = {10, 20, 20, 20, 30, 40, 50};

// equal_range 返回一个pair
// first: lower_bound的位置
// second: upper_bound的位置
auto range = equal_range(v.begin(), v.end(), 20);

cout << "等于20的范围: " << endl;
for(auto it = range.first; it != range.second; ++it) {
    cout << *it << " "; // 20 20 20
}
```



```

cout << endl;

// 必须先用 sort 排序!
sort(v.begin(), v.end());

// 1. upper_bound(开始, 结束, 值)
auto it = upper_bound(v.begin(), v.end(), 20);
if(it != v.end()) {
    cout << "第一个 > 20 的位置: " << it - v.begin() << endl; //
4
    cout << "该位置的值: " << *it << endl; // 30
}

// 2. 查找不存在的值
auto it2 = upper_bound(v.begin(), v.end(), 25);
cout << "第一个 > 25 的值: " << *it2 << endl; // 30

// 3. 配合 lower_bound
auto lb = lower_bound(v.begin(), v.end(), 20); // 第一个 >= 20
auto ub = upper_bound(v.begin(), v.end(), 20); // 第一个 > 20
cout << "20的个数: " << ub - lb << endl; // 3

```

- 计数: count

3.2.2 set集合

- 默认排序, 默认升序, 默认没有重复元素。(可以用于排序)

```

set<int> s = {5, 2, 8, 1, 9, 2, 5}; //也可以复制构造函数等
for(int num : s) {
    cout << num << " "; // 1 2 5 8 9
}

```

- 操作具体如下:

```

set<int> s; //如果希望降序, 则set<int, greater<string>>, 或者希望自己的规则, 定义比较函数
/* 不能用普通函数作为比较函数在这里!
struct CompareLength {
    bool operator()(const string& a, const string& b) const {
        if(a.length() != b.length()) {
            return a.length() < b.length(); // 先按长度升序
        }
        return a < b; // 长度相同按字典序
    }
};

```

```

*/

// 1. insert 返回值
auto ret1 = s.insert(10); // 插入10
cout << "插入10: " << (ret1.second ? "成功" : "失败") << endl;
cout << "位置的值: " << *ret1.first << endl;

auto ret2 = s.insert(10); // 再次插入10
cout << "再次插入10: " << (ret2.second ? "成功" : "失败") << endl;

// 2. 插入多个值
s.insert({20, 30, 40, 50});

// 3. 提示插入位置
auto hint = s.lower_bound(25); // 第一个不小于
s.insert(hint, 25); // 提示位置, 提高效率

// 4. 插入迭代器范围
vector<int> v = {15, 35, 45};
s.insert(v.begin(), v.end());

cout << "最终set: ";
for(int n : s) cout << n << " "; // 10 15 20 25 30 35 40 45 50
cout << endl;

// 1. find - 查找元素是否存在
auto it = s.find(30);
if(it != s.end()) {
    cout << "找到30: " << *it << endl;
} else {
    cout << "没找到30" << endl;
}

// 查找不存在的
it = s.find(35);
if(it == s.end()) {
    cout << "35不存在" << endl;
}

// 2. count - 统计出现次数 (set中只有0或1)
if(s.count(40) > 0) {
    cout << "40存在" << endl;
}

// 3. 范围查找
int target = 45;
auto lb = s.lower_bound(target); // 第一个>=45
auto ub = s.upper_bound(target); // 第一个>45

```

```

if(lb != s.end()) {
    cout << "第一个>=45: " << *lb << endl; // 50
}
if(ub != s.end()) {
    cout << "第一个>45: " << *ub << endl; // 50
}

// 4. equal_range
auto range = s.equal_range(30);
if(range.first != range.second) {
    cout << "等于30的元素个数: " << distance(range.first,
range.second) << endl;
}

// 1. erase(值) - 返回删除个数
int n = s.erase(30);
cout << "删除了" << n << "个元素" << endl;

// 删除不存在的
n = s.erase(35);
cout << "删除35的结果: " << n << endl; // 0

// 2. erase(迭代器)
auto it = s.find(40);
if(it != s.end()) {
    s.erase(it);
}

// 3. erase(迭代器范围)
auto first = s.lower_bound(50);
auto last = s.upper_bound(80);
s.erase(first, last); // 删除[50, 80]

cout << "删除后: ";
for(int x : s) cout << x << " "; // 10 20 90
cout << endl;

// 4. clear
s.clear();
cout << "清空后大小: " << s.size() << endl;

//特殊操作merge
s1.merge(s2);

```

3.2.3 map映射

- 默认排序，默认升序，默认没有重复元素。（可以用于排序）

```
// 1. 空map
map<string, int> m1;

// 2. 初始化列表
map<string, int> m2 = {
    {"Alice", 90},
    {"Bob", 85},
    {"Charlie", 92}
};

// 1. insert 插入
auto ret1 = m.insert({"Alice", 90});
cout << "插入Alice: " << (ret1.second ? "成功" : "失败") << endl;

// 插入已存在的key
auto ret2 = m.insert({"Alice", 95}); // 不会修改
cout << "再次插入Alice: " << (ret2.second ? "成功" : "失败") << endl;

// 2. 下标插入（会自动创建）
m["Bob"] = 85; // 插入Bob
m["Bob"] = 88; // 修改Bob的分数

// 3. insert多种方式
m.insert(make_pair("Charlie", 92));
m.insert(pair<string, int>("David", 88));

// 4. 提示插入位置
auto hint = m.lower_bound("E");
m.insert(hint, {"Eve", 95}); // 提示位置

// 5. insert多个
map<string, int> temp = {"Frank", 82}, {"Grace", 91};
m.insert(temp.begin(), temp.end());

// 1. 下标访问
cout << "Alice的成绩: " << scores["Alice"] << endl; // 90

// 2. 修改
scores["Alice"] = 95; // 修改

// 3. 注意：下标访问会自动插入
cout << "不存在的David: " << scores["David"] << endl; // 0, 自动创建
cout << "map大小: " << scores.size() << endl; // 4
```

```

// 4. 安全访问
auto it = scores.find("Eve");
if(it != scores.end()) {
    cout << "Eve的成绩: " << it->second << endl;
} else {
    cout << "Eve不存在" << endl;
}

// 1. find
auto it = scores.find("Charlie");
if(it != scores.end()) {
    cout << "找到Charlie: " << it->second << endl;
}

// 2. count
if(scores.count("Bob") > 0) {
    cout << "Bob存在" << endl;
}

// 3. lower_bound/upper_bound
auto low = scores.lower_bound("C"); // 第一个>=C
auto high = scores.upper_bound("E"); // 第一个>E

// 1. erase(键)
int n = scores.erase("Bob");
cout << "删除Bob: " << (n > 0 ? "成功" : "失败") << endl;

// 2. erase(迭代器)
auto it = scores.find("David");
if(it != scores.end()) {
    scores.erase(it);
}

// 3. erase(范围)
auto first = scores.lower_bound("C");
auto last = scores.upper_bound("F");
scores.erase(first, last);

// 4. clear
scores.clear();
cout << "清空后大小: " << scores.size() << endl;

```

3.2.4 多重集合（multiset）与多重映射（multimap）

- 多重集合与集合、多重映射与映射的用法差不多，只在几个成员函数上有细微差异，其差异主要表现在去除了键必须唯一的限制。

4 函数对象

- lambda表达式
- 用于算术运算的函数对象：
 - 一元函数对象：negate
 - 二元函数对象：plus、minus、multiplies、divides、modulus
- 用于关系运算、逻辑运算的函数对象
 - 一元谓词：logical_not
 - 二元谓词：equal_to、not_equal_to、greater、less、greater_equal、less_equal、logical_and、logical_or

5 算法

- 不可变序列算法
 - 不直接修改所操作的容器内容的算法
 - 用于查找指定元素、比较两个序列是否相等、对元素进行计数等
- 可变序列算法
 - 可以修改它们所操作的容器对象
 - 包括对序列进行复制、删除、替换、倒序、旋转、交换、变换、分割、去重、填充、洗牌的算法及生成一个序列的算法
- 排序和搜索算法
 - 对序列进行排序
 - 对两有序序列进行合并
 - 对有序序列进行搜索
 - 有序序列的集合操作
 - 堆算法

5.1 排序用 `sort`

```
#include <algorithm>
#include <vector>
#include <iostream>
using namespace std;

int main() {
```

```

vector<int> v = {5, 2, 8, 1, 9};

// 默认升序
sort(v.begin(), v.end()); // 1 2 5 8 9

// 降序
sort(v.begin(), v.end(), greater<int>()); // 9 8 5 2 1

// 自定义排序
sort(v.begin(), v.end(), [](int a, int b) {
    return a > b; // 降序
});

// 按绝对值排序
sort(v.begin(), v.end(), [](int a, int b) {
    return abs(a) < abs(b);
});

return 0;
}

```

5.2 查找用 `find`

```

#include <algorithm>
#include <vector>
#include <iostream>
using namespace std;

int main() {
    vector<int> v = {1, 2, 3, 4, 5};

    // 查找元素
    auto it = find(v.begin(), v.end(), 3);
    if(it != v.end()) {
        cout << "找到3, 位置: " << it - v.begin() << endl; // 2
    } else {
        cout << "没找到" << endl;
    }

    // 在字符串中查找
    string s = "hello world";
    auto it2 = find(s.begin(), s.end(), 'w');

    return 0;
}

```

5.3 累加用 `accumulate`

```
#include <numeric> // 注意头文件
#include <vector>
#include <iostream>
using namespace std;

int main() {
    vector<int> v = {1, 2, 3, 4, 5};

    // 求和
    int sum = accumulate(v.begin(), v.end(), 0);
    cout << "和: " << sum << endl; // 15

    // 求积
    int product = accumulate(v.begin(), v.end(), 1,
        [](int a, int b) { return a * b; });
    cout << "积: " << product << endl; // 120

    return 0;
}
```

5.4 转换用 `transform`

```
#include <algorithm>
#include <vector>
#include <iostream>
using namespace std;

int main() {
    vector<int> v = {1, 2, 3, 4, 5};
    vector<int> result(v.size());

    // 每个元素平方
    transform(v.begin(), v.end(), result.begin(),
        [](int x) { return x * x; });
    // result: 1 4 9 16 25

    // 就地转换
    transform(v.begin(), v.end(), v.begin(),
        [](int x) { return x + 10; });
    // v: 11 12 13 14 15
}
```



```
    return 0;
}
```

5.5 反转用 `reverse`

```
#include <algorithm>
#include <vector>
#include <iostream>
using namespace std;

int main() {
    vector<int> v = {1, 2, 3, 4, 5};

    // 反转整个容器
    reverse(v.begin(), v.end()); // 5 4 3 2 1

    // 反转部分
    reverse(v.begin() + 1, v.end() - 1); // 5 2 3 4 1

    // 字符串反转
    string s = "hello";
    reverse(s.begin(), s.end()); // "olleh"

    return 0;
}
```

5.6 计数用 `count`

```
#include <algorithm>
#include <vector>
#include <iostream>
using namespace std;

int main() {
    vector<int> v = {1, 2, 2, 3, 3, 3, 4, 4, 5};

    // 统计2出现的次数
    int cnt = count(v.begin(), v.end(), 2);
    cout << "2出现了" << cnt << "次" << endl; // 2

    // 统计偶数个数
    int even_cnt = count_if(v.begin(), v.end(),
```

```

        [](int x) { return x % 2 == 0; });
    cout << "偶数个数: " << even_cnt << endl; // 4

    return 0;
}

```

5.7 去重用 `unique` + `sort`

```

#include <algorithm>
#include <vector>
#include <iostream>
using namespace std;

int main() {
    vector<int> v = {1, 2, 2, 3, 3, 3, 4, 4, 5};

    // 1. 先排序
    sort(v.begin(), v.end());

    // 2. 去重, 返回去重后的结束位置
    auto new_end = unique(v.begin(), v.end());

    // 3. 删除重复元素
    v.erase(new_end, v.end());

    for(int n : v) cout << n << " "; // 1 2 3 4 5
    cout << endl;

    return 0;
}

```

5.8 最值用 `max/min_element`

```

#include <algorithm>
#include <vector>
#include <iostream>
using namespace std;

int main() {
    vector<int> v = {5, 2, 8, 1, 9, 3};

    // 最大值

```

```

auto max_it = max_element(v.begin(), v.end());
cout << "最大值: " << *max_it
      << " 位置: " << max_it - v.begin() << endl;

// 最小值
auto min_it = min_element(v.begin(), v.end());
cout << "最小值: " << *min_it
      << " 位置: " << min_it - v.begin() << endl;

// 多个数比较
cout << "max(3,5,1): " << max({3, 5, 1}) << endl; // 5
cout << "min(3,5,1): " << min({3, 5, 1}) << endl; // 1

return 0;
}

```

5.9 填充用 `fill`

```

#include <algorithm>
#include <vector>
#include <iostream>
using namespace std;

int main() {
    vector<int> v(5); // 5个0

    // 全部填充为-1
    fill(v.begin(), v.end(), -1);
    // v: -1 -1 -1 -1 -1

    // 部分填充
    fill(v.begin() + 1, v.end() - 1, 0);
    // v: -1 0 0 0 -1

    // 二维数组填充
    vector<vector<int>> matrix(3, vector<int>(4));
    for(auto& row : matrix) {
        fill(row.begin(), row.end(), 1);
    }

    return 0;
}

```

5.10 复制用 `copy`

```
#include <algorithm>
#include <vector>
#include <iostream>
#include <iterator> // back_inserter
using namespace std;

int main() {
    vector<int> src = {1, 2, 3, 4, 5};
    vector<int> dest1(src.size());

    // 普通复制
    copy(src.begin(), src.end(), dest1.begin());

    // 复制到输出流
    copy(src.begin(), src.end(), ostream_iterator<int>(cout, " "));
    cout << endl; // 1 2 3 4 5

    // 复制时筛选
    vector<int> dest2;
    copy_if(src.begin(), src.end(), back_inserter(dest2),
        [](int x) { return x % 2 == 0; }); // 只复制偶数
    // dest2: 2 4

    return 0;
}
```

5.11 合并用 `merge`

```
#include <algorithm>
#include <vector>
#include <iostream>
using namespace std;

int main() {
    // 两个已排序的数组，必须先排好序
    vector<int> v1 = {1, 3, 5, 7, 9};
    vector<int> v2 = {2, 4, 6, 8, 10};
    vector<int> result(v1.size() + v2.size());

    // 合并到result
    merge(v1.begin(), v1.end(),
        v2.begin(), v2.end(),
        result.begin());
}
```

```
cout << "合并结果: ";  
for(int n : result) cout << n << " "; // 1 2 3 4 5 6 7 8 9 10  
cout << endl;  
  
return 0;  
}
```

第十一章：流类库与输入输出

1 概念

- 程序对象 —— 流对象 —— 文件对象（包括屏幕cout/cin、磁盘文件fstream、字符串模拟stringstream）；
- 从流中读取，称之提取 `cin >> x`；向流中写入，称之插入 `cout << x`；
- 头文件：`#include<iostream>`，`#include<fstream>`，`#include<stringstream>`。

2 输出流

- 继承关系如下：

```
ostream (基本输出流)
  ↓
ofstream (文件输出流)
  ↓
ostringstream (字符串输出流)
```

2.1 ostream

- cout是他的一个实例，操纵符如下。

2.2 ofstream

- 纯文本文件：
 - `ofstream ofs("./path", ios::out);`
 - `ofs.close();`
- 二进制文件：
 - `ofstream ofs("./path", ios::out | ios::binary);`
 - `ofs.write(reinterpret_cast<char *> (&stu), sizeof(stu));`
 - `ofs.close();`
- 操纵符（操纵算子）：
 - `#include <iomanip>`
 - `setw(10)` 和 `cout.width(10)` 指定宽度，其他格式符保持直到发生改变。
 - 对齐：默认右对齐，可以用 `setiosflags(ios_base::left())` 和 `resetiosflags(ios_base::left())`。还有很多ios_base的参数，这里不做展示，可以用或运算符同时设置。
 - `dec, oct, hex;`

- 如果没有设置 `ios_base::fixed` 和 `ios_base::scientific`, `setprecision` 表示有效位; 否则表示小数位数。

2.3 ostringstream

- 可以用于生成一个string变量, 具体例子在后面。

3 输入流

- 继承关系如下:

```
istream (基本输入流)
  ↓
ifstream (文件输入流)
  ↓
istringstream (字符串输入流)
```

3.1 istream

- cin是他的一个实例;
- cin.get(); 和 getline()的用法示例: EOF是get()重要的标志符。

```
#include <iostream>
#include <cstring>

using namespace std;

int main(){
    /*
    i am ldk    //\n, input
    i
    ,          //\n, input
    am ldk

    e          //\n, input
    e
    */

    string str1, str2;
    cin >> str1;    // don't eat '\n'
    cout << str1 << endl;
    cin.ignore();

    getline(cin, str2, ','); // eat ','
```

```

    cout << str2 << endl;
    cin.ignore();

    char tmp = cin.get();
    cout.put(tmp);

    return 0;
}

```

3.2 ifstream

- 纯文本文件:

- `ifstream ifs("./path", ios::in);`
- `ifs.close();`

- 二进制文件:

- `ifstream ifs("./path", ios::in | ios::binary);`
- `ifs.read(reinterpret_cast<char *> (&stu), sizeof(stu));`
- `ofs.close();`

- 综合案例:

```

#include <iostream>
#include <cstring>
#include <fstream>
using namespace std;

class Student {
private:
    char name; // can not be string!
    int id;
public:
    Student(){}
    Student(char n, int i) : name(n), id(i){}
    ~Student(){}
    void print_info(){
        cout << name << ":" << id << endl;
    }
};

int main(){
    Student stu[3] = {{'a', 1}, {'b', 2}, {'c', 3}};
    cout << "init:" << endl;
    for(int i = 0; i < 3; i++){
        stu[i].print_info();
    }
}

```



```

    }

    ofstream ofs("./binaryfile", ios::out | ios::binary);
    if(!ofs){
        cout << "error in creating binaryfile!" << endl;
        return -1;
    }
    ofs.write(reinterpret_cast<char *>(&stu), sizeof(stu));
    ofs.close();

    Student other_stu[3];
    ifstream ifs("./binaryfile", ios::in | ios::binary);
    if(!ifs){
        cout << "error in opening binaryfile!" << endl;
        return -1;
    }
    ifs.read(reinterpret_cast<char *>(&other_stu),
sizeof(other_stu));
    ifs.close();

    cout << "after:" << endl;
    for(int i = 0; i < 3; i++){
        other_stu[i].print_info();
    }

    return 0;
}

```

```

#include <iostream>
#include <fstream>
#include <sstream>
#include <iomanip>
#include <cstring>
#include <vector>
using namespace std;

int main(){
    // read text from input.txt, store to vector<string>.
    ifstream input_ifs("./input.txt", ios::in | ios::binary);
    if(!input_ifs){
        cout << "failed to open!" << endl;
        return -1;
    }
    vector<string> x;
    string tmp;
    while(getline(input_ifs, tmp)){
        cout << "origin text:" << tmp << endl;
    }
}

```

```

        x.push_back(tmp);
    }
    input_ifs.close();

    // use binary mode to deal with text is a good choice!
    ofstream output_ofs("./output1", ios::out | ios::binary);
    if(!output_ofs){
        cout << "failed to create output1!" << endl;
        return -1;
    }
    for(int i = 0; i < x.size(); i++){
        output_ofs << i+1 << ": " << x[i] << endl;
    }
    output_ofs.close();

    // use text mode
    output_ofs.open("./output2.txt", ios::out);
    if(!output_ofs){
        cout << "failed to create output2!" << endl;
        return -1;
    }
    for(int i = 0; i < x.size(); i++){
        output_ofs << i+1 << ": " << x[i] << endl;
    }
    output_ofs.close();
    return 0;
}

```

3.3 istreamstringstream

- 可以用于拆解一个string变量;
- 综合案例:

```

#include <iostream>
#include <sstream>
#include <cstring>
using namespace std;
int main(){
    /*
        << means getting in a stream, used to output!
        >> means getting things from stream, used to input!
        HERE: we view string as your screens!
    */
    //ostringstream, create string!
    ostringstream oss;
    oss << 5 << 'a' << "happy";
    cout << oss.str() << endl;
}

```

```
//istringstream, seperate string!  
string str = "1.2 5";  
istringstream iss(str);  
double x, y;  
iss >> x >> y;  
cout << x << y << endl; // 1.25  
  
return 0;  
}
```

4 输入输出流

- 了解即可，使用的时候不建议。建议直接分为两个。
- `iostream`, `fstream`, `stringstream`这三个！

第十二章：异常处理

```
#include <iostream>
#include <cstring>
#include <stdexcept> // for invalid_argument
using namespace std;

class MyException{
public:
    string error;
    MyException(string str):error(str){}
    ~MyException(){}
    string getReason(){
        return error;
    }
};

int my_div(int x, int y) /*throw(MyException) c++17abort!*/{
    if(y == 0) throw MyException("dividing a zero!");
    return x/y;
}

int div1(int x, int y){
    if(y == 0) throw invalid_argument("dividing a zero!");
    return x/y;
}

int main(){
    try{
        cout << my_div(10, 3) << endl;
        cout << my_div(6, 0) << endl;
        cout << my_div(5, 2) << endl;
    }catch(MyException &e){ //recommmended
        cout << e.getReason() << endl;
        /* MyException e : copy!
           MyException : also works! */
    }

    try{
        cout << div1(10, 3) << endl;
        cout << div1(6, 0) << endl;
        cout << div1(5, 2) << endl;
    }catch(exception &e){
        /* & is necessary for polymorphism
           can change to invalid_argument! */
    }
}
```

```
        cout << e.what() << endl;
    }
    return 0;
}
```